

AI/ML, LLM Integration, React & Modern Frameworks



High performance. Delivered.

consulting | technology | outsourcing



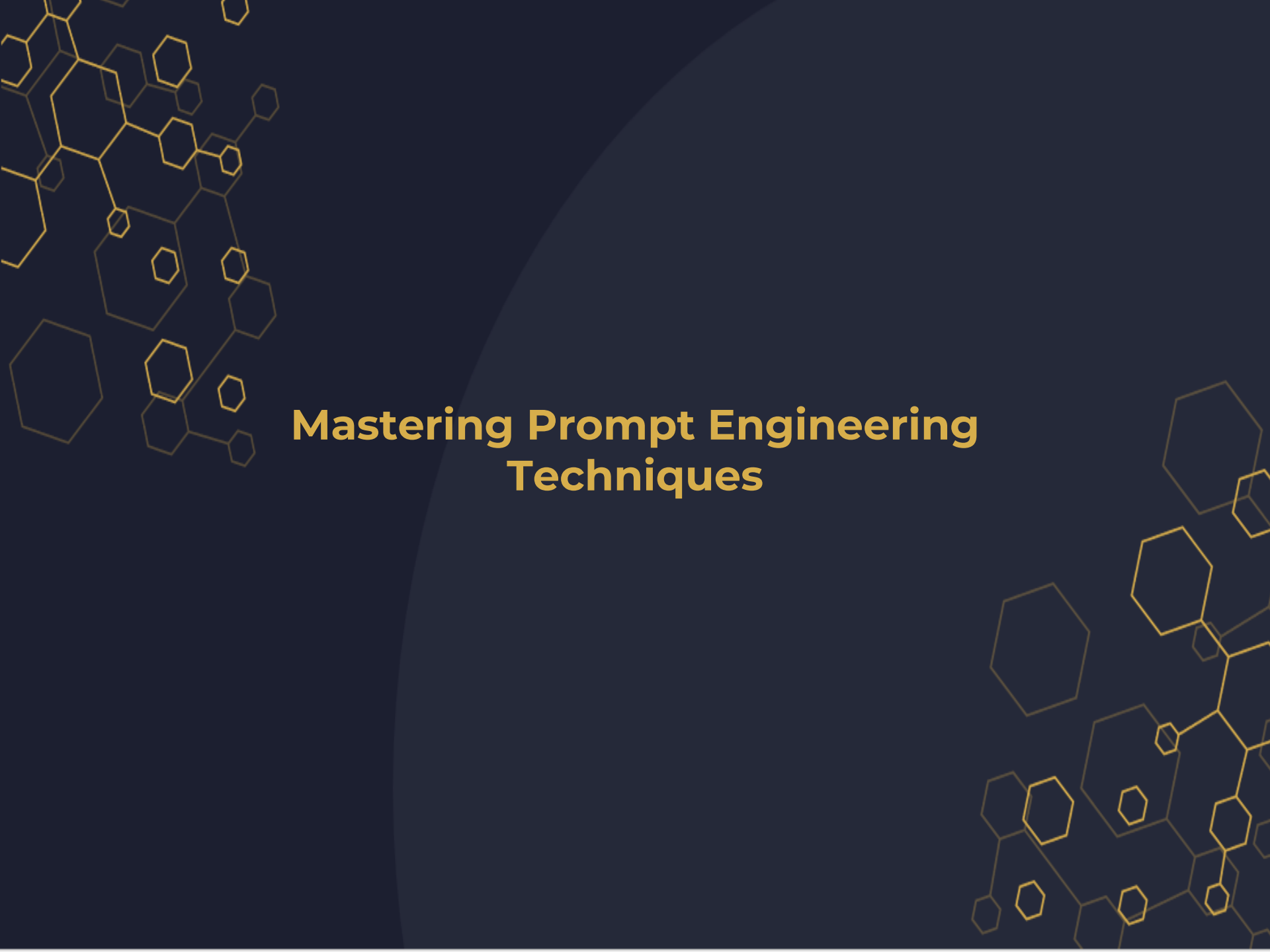
Goals

- Master **advanced prompt engineering** techniques for reliable and effective LLM interactions.
- Understand **AI agents**, planning, memory, tools, ReAct patterns, and autonomous workflows.
- Gain hands-on knowledge of **MCP architecture**, tools, resources, context management, skills, and plugins.
- Integrate applications with **OpenAI, Azure OpenAI, Google Gemini/ADK, and BMW GAIA**.
- Build AI applications using **LangChain and LangGraph** with chains, agents, memory, and stateful workflows.
- Design and implement **RAG solutions** using embeddings, vector databases, retrieval, re-ranking, and hybrid search.
- Develop **tool-enabled and multi-step agentic AI applications** for enterprise use cases.
- Apply **responsible AI, hallucination reduction, token optimization, security, and cost-awareness**.

Goals



- Build and deploy ML/AI solutions using **AWS SageMaker** across training, tuning, deployment, and monitoring.
- Develop scalable APIs using **FastAPI**, Pydantic, async programming, WebSockets, and OpenAPI.
- Use **Supabase/PostgreSQL** for database, authentication, storage, real-time data, and security.
- Automate AI and business workflows using **n8n**, APIs, databases, schedules, and event-driven integrations.
- Understand **A2A communication**, REST APIs, webhooks, message queues, and asynchronous architectures.
- Build AI/data application frontends using **React and Angular**.
- Apply automated testing using **pytest, unittest, Jest, mocking, and coverage**.
- Follow **clean code, refactoring, code reviews, documentation, Git, and software engineering best practices**.
- Enable participants to build **end-to-end, production-ready Generative AI and agentic solutions** independently.



Mastering Prompt Engineering Techniques



What is Generative AI?

- **Generative AI (GenAI)** is a type of artificial intelligence that learns patterns from large amounts of data and can **generate new content** based on user instructions or prompts.
- It can generate or transform:
 - **Text** – reports, summaries, emails, documentation
 - **Code** – Python, Java, SQL, APIs, unit tests
 - **Images** – vehicle concepts, marketing visuals
 - **Audio and speech**
 - **Video**
 - **Structured data**
 - **Recommendations and explanations**

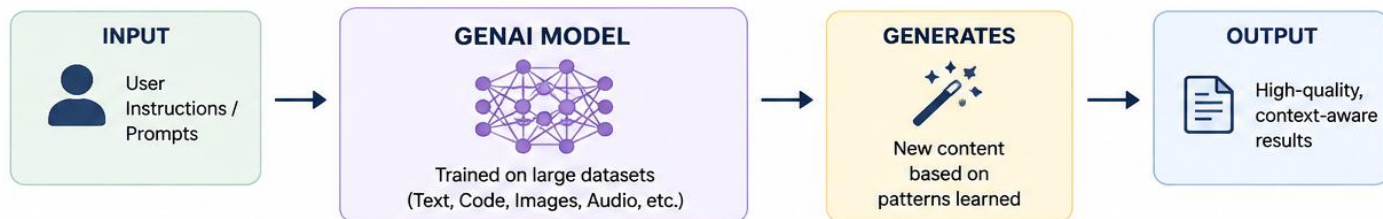
What is Generative AI?



Generative AI (GenAI) – What It Is & What It Can Generate



Generative AI (GenAI) is a type of artificial intelligence that **learns patterns** from large amounts of data and can **generate new content** based on user instructions or prompts.



WHAT GENAI CAN GENERATE OR TRANSFORM



GENAI IN BMW – REAL WORLD APPLICATIONS





What is Generative AI?

- For example:
- **Prompt:**
“Analyze this BMW vehicle telemetry and explain why the battery temperature is increasing.”
- **GenAI Response:**
The model can interpret the telemetry, summarize possible causes, highlight unusual values, and recommend what an engineer should investigate.



Real-Time Generative AI Use Cases in BMW

- **1. Vehicle Diagnostic Assistant**
- Vehicle telemetry such as:
- Vehicle → Battery → Temperature → Fault Codes → GenAI
- GenAI can:
 - Explain fault codes in natural language.
 - Summarize vehicle health.
 - Suggest troubleshooting steps.
 - Generate technician repair instructions.
 - Compare current telemetry with historical incidents.
- **Example:**
“BMW i4 battery temperature reached 88°C. Explain possible causes and recommended actions.”



- **2. Predictive Maintenance Assistant**
- BMW vehicles continuously generate sensor data.
- **Vehicle Sensors → Data Platform → ML/Analytics → GenAI → Service Advisor**
- GenAI can convert technical predictions into understandable recommendations such as:
- “Vehicle BMW1002 may require battery cooling-system inspection within the next service cycle because temperature readings have increased consistently.”



Real-Time Generative AI Use Cases in BMW

- **3. Manufacturing Quality Assistant**
- Production-line data can include:
 - Robot data
 - Welding information
 - Paint quality
 - Torque measurements
 - Camera inspection results
 - Equipment alarms
- GenAI can analyze the information and provide explanations.
- **Example question:**
- “Why has the defect rate increased on Assembly Line 3 today?”
- The AI assistant could combine available production data and generate a summarized root-cause investigation.



Real-Time Generative AI Use Cases in BMW

- **4. Engineering Knowledge Assistant**
- BMW engineers may have thousands of documents such as:
 - Design specifications
 - Technical manuals
 - Maintenance documentation
 - Engineering standards
 - Historical problem reports
- Using **RAG + Generative AI**:
- **Engineer Question → Search BMW Documents → Retrieve Relevant Information → LLM → Answer**
- Example:
- “What are the recommended diagnostic steps for an iX battery cooling-system fault?”



Real-Time Generative AI Use Cases in BMW

- **5. Software Developer Assistant**
- BMW software teams can use GenAI to assist with:
 - Code generation
 - Unit-test generation
 - Code explanation
 - Refactoring
 - API development
 - SQL generation
 - Documentation
- For example:
- **Developer:**
 - “Create a FastAPI endpoint to retrieve vehicle telemetry.”
- GenAI can generate the API, validation models, tests, and documentation.



Real-Time Generative AI Use Cases in BMW

- **6. Customer Service Assistant**
- A BMW conversational assistant could handle questions such as:
 - “How do I schedule my next service?”
 - “Explain this dashboard warning.”
 - “Where is the nearest service center?”
 - “How can I improve my EV range?”
 - “Explain my vehicle features.”
- GenAI provides a **natural-language interface** instead of requiring customers to navigate multiple systems.



Real-Time Generative AI Use Cases in BMW

- **7. Sales & Vehicle Recommendation**
- GenAI can help customers compare vehicles.
- For example:
 - “I travel 60 km every day and want an electric BMW with good range and family space. Which models should I consider?”
- The system can combine customer requirements with approved product information to generate recommendations.



Real-Time Generative AI Use Cases in BMW

- **8. Supply-Chain Assistant**
- GenAI can summarize:
 - Supplier delays
 - Parts shortages
 - Inventory
 - Production impact
 - Logistics issues
- Example:
- “Which production lines could be affected if supplier ABC delays battery modules by three days?”
- The assistant can summarize operational data and support planners' decisions.



Real-Time Generative AI Use Cases in BMW

- **9. Automated Business Reports**
- Instead of manually analyzing dashboards, GenAI can transform business data into management summaries.
- For example:
 - **BMW Sales Database → Analytics → GenAI**
- Output:
- “EV sales increased compared with the previous period, with the i4 contributing strongly. Munich and Berlin showed the largest regional growth.”



Real-Time Generative AI Use Cases in BMW

- **10. Intelligent In-Vehicle Assistant**
- Generative AI can make an in-car assistant more conversational.
- A driver could say:
 - “I’m getting tired. Find a charging station with a coffee shop on my route and explain how much battery I’ll have when I arrive.”
- The AI system can combine **natural-language understanding + navigation + vehicle data + external services** to respond.



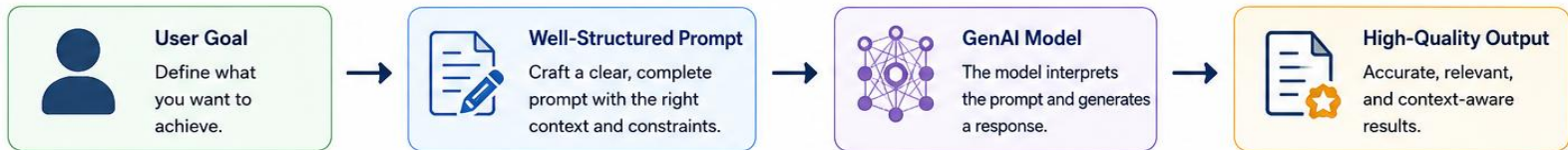
Prompt Engineering

- **Prompt Engineering** is the process of designing and refining instructions given to a Generative AI model so that it produces **accurate, relevant, consistent, and useful responses**.
- A prompt typically tells the model:
 - **What role to take**
 - **What task to perform**
 - **What context to use**
 - **What constraints to follow**
 - **What output format to return**

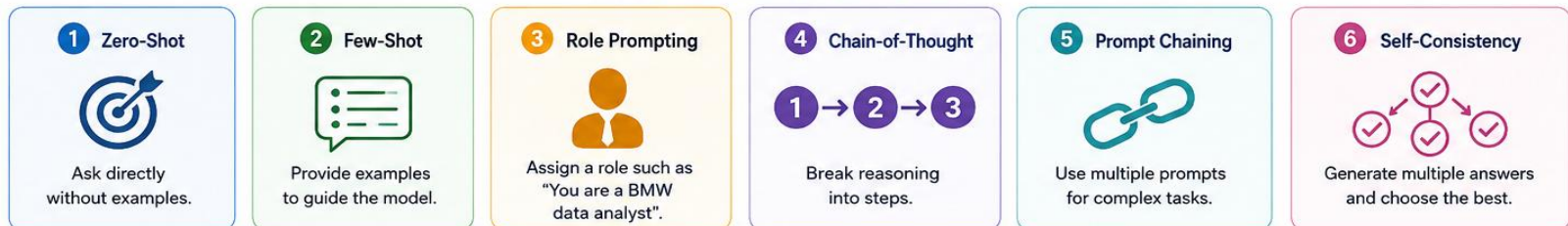
Prompt Engineering



Prompt Engineering – Designing Better Prompts for Better AI Output



CORE PROMPTING TECHNIQUES



BMW USE CASES





Prompt Engineering

- **Simple BMW Example**
- **Weak prompt:**
“Analyze BMW data.”
- **Better prompt:**
“You are a BMW vehicle diagnostic expert. Analyze the telemetry data for battery level, temperature, speed, and fault codes. Identify abnormal values, explain possible causes, and return the result as a table with Severity, Issue, Cause, and Recommended Action.”

Prompt Engineering Tools



Interactive LLM Tools	ChatGPT	Write, refine and test prompts interactively	Conversational interface for iterating and improving prompts
	OpenAI Playground	Experiment with OpenAI models and prompt settings	Test models, parameters (temperature, top-p, etc.) and compare outputs
	Claude / Anthropic Console	Design and experiment with Claude prompts	Fine-tune prompts and explore Claude model behavior
	Google AI Studio	Develop prompts for Gemini models	Build, test and evaluate prompts for Gemini models
Enterprise AI Platforms	Vertex AI Studio	Enterprise Gemini prompt development on Google Cloud	Build, evaluate and deploy prompts at enterprise scale
	Microsoft Foundry	Prototype prompts, models and AI agents on Azure	End-to-end platform for models, prompt flows and AI agents
Coding Assistants	GitHub Copilot Chat	Prompt-based coding and development assistance	Generate, explain, refactor and debug code using prompts
	Gemini Code Assist	AI-assisted coding and prompt-based development	Code generation, completion, explanations and refactoring
	Claude Code	Prompt-driven software development workflows	Agentic coding, refactoring, testing and documentation
Development & Experimentation	VS Code	Develop and test prompts inside applications	Integrate, test and run prompts in your apps and extensions
	Jupyter Notebook	Experiment with prompts using Python and APIs	Run experiments, analyze results and build prompt pipelines

Prompt Engineering Tools



Prompt Engineering Ecosystem Tools

Tools for prompt testing, management, evaluation, tracing, orchestration, and optimization

1. Testing, Evaluation & Tracing



LangSmith

Prompt testing, versioning, evaluation and tracing



Humanloop

Prompt experimentation and evaluation



Promptfoo

Automated prompt testing and comparison



Weights & Biases Weave

LLM experiments, tracing and evaluation



Arize Phoenix

LLM tracing, evaluation and RAG analysis



Parea

Prompt experimentation and evaluation

2. Prompt Management & Observability



PromptLayer

Prompt management, tracking and versioning



Langfuse

Open-source prompt management and LLM observability



Helicone

LLM monitoring, analytics and prompt tracking

3. Prompt Templates, Orchestration & Workflow



LangChain PromptTemplate

Reusable prompts with variables



Semantic Kernel

Prompt templates, functions and AI orchestration



LangGraph

Prompt-driven stateful agent workflows

4. Optimization & Programmatic Prompting



DSPy

Programmatic prompt optimization

Where These Tools Help



Test prompts



Track versions



Evaluate outputs



Monitor LLM apps



Build agent workflows

Common Prompt Engineering Techniques

Type	Meaning	BMW Example
Zero-Shot	Perform a task without examples.	“Classify this BMW fault as Low, Medium, or High.”
Few-Shot	Provide examples before asking the model to respond.	“90°C → High, low tire pressure → Medium. Classify 78°C.”
Structured Reasoning	Analyze a problem systematically before giving a conclusion.	“Analyze temperature, speed, and fault codes and give likely cause.”
Tree of Thoughts	Explore multiple possible solutions and select the best one.	Compare cooling failure, sensor error, and high-load driving.
Prompt Chaining	Break a complex task into multiple connected prompts.	Detect anomalies → Identify causes → Recommend maintenance.



Common Prompt Engineering Techniques

Type	Meaning	BMW Example
Self-Consistency	Compare multiple candidate answers and select the most reliable.	Evaluate multiple diagnostic possibilities and choose the strongest result.
Role Prompting	Assign the model a specific expert role.	"You are a BMW vehicle diagnostic expert."
Instruction Following	Clearly define rules, constraints, and output format.	"Use only telemetry data and return Issue, Severity, and Action."
Hallucination Reduction	Restrict answers to trusted information.	"Use only the BMW service manual; say 'Insufficient information' if unavailable."
Iterative Refinement	Improve the prompt based on previous responses.	"Analyze vehicle data" → add battery, severity, cause, and output format.

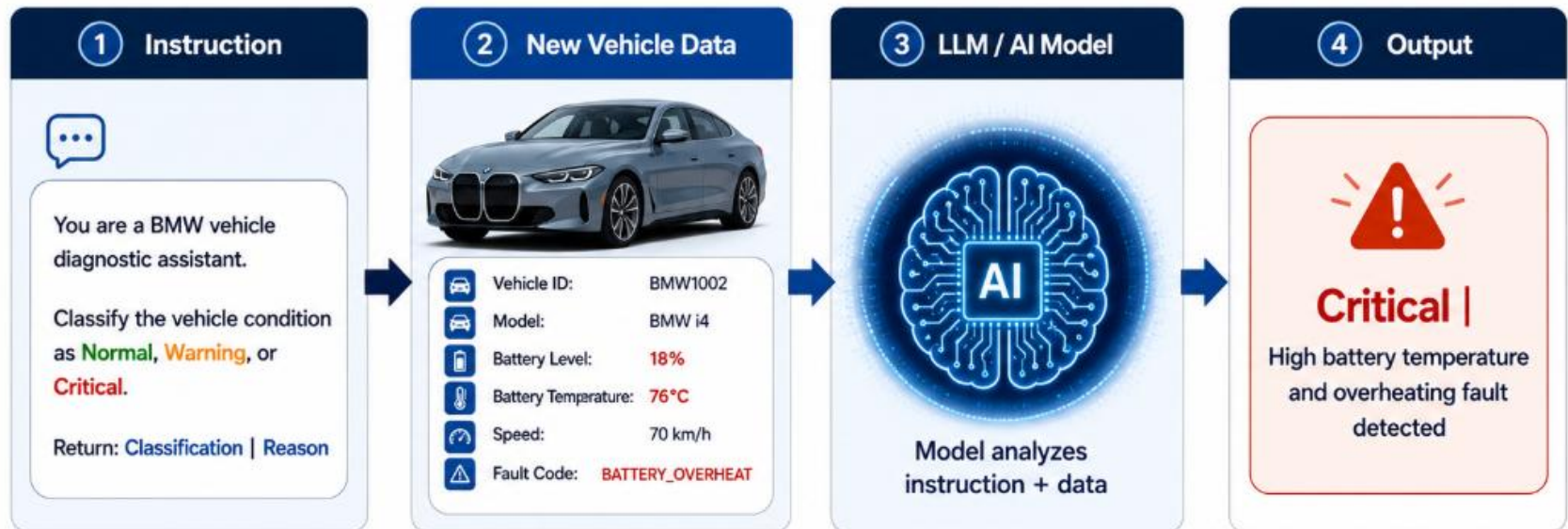


Why Is This Zero-Shot?

- The model is given:
- **Instruction + Vehicle Data + Expected Output Format**
- The model is **not given any previous examples** showing how to classify BMW vehicle conditions.
- Therefore, it must perform the task directly from the instruction.
- **Simple Formula**
- **Zero-Shot = Instruction + Input Data → AI Response**
- No examples are provided.

Zero-Shot Prompting – BMW Vehicle Diagnostic Example

Instruction + New Input → AI Response (No examples provided)



Why is this Zero-Shot?



No examples are given in the prompt



The model uses only the instruction and input data



It directly produces the classification



Use case:
Real-time BMW vehicle health monitoring

Zero-Shot Prompting – Hands-On

- **Scenario: BMW Vehicle Health Alert**
- A BMW i4 is travelling on the highway. The connected-vehicle platform receives live telemetry from the car.
- The monitoring system detects unusual battery conditions and sends the following data to an AI assistant:
 - Vehicle ID: BMW1002
 - Model: BMW i4
 - Battery Level: 15%
 - Battery Temperature: 88°C
 - Vehicle Speed: 70 km/h
 - Fault Code: BATT_TEMP_HIGH



Zero-Shot Prompting – Hands-On

- **Business Requirement**
- The BMW service team wants Generative AI to quickly analyze the telemetry and answer:
 - Is there a problem with the vehicle?
 - How serious is the problem?
 - What could be the possible cause?
 - What should the driver or service team do next?
 - No sample diagnosis is given to the model.



Few-Shot Prompting with examples

- **Few-shot prompting** means giving the model a **small number of examples** before asking it to perform a similar task.
- The examples help the model understand:
 - the expected pattern,
 - the type of answer,
 - the output format,
 - and the classification logic.

Few-Shot Prompting with examples



Few-Shot Prompting in BMW Use Cases



Learning from a small number of examples



Few-shot prompting helps an AI model perform a task by learning the pattern from a small set of examples.

Benefits for BMW



Rapid adaptability



Reduced data needs



Better accuracy for domain tasks



Faster AI solution development

How Few-Shot Prompting Works – BMW Example

Examples



Engine Overheat



Low Battery



Low Tire Pressure



New Vehicle Input



High engine temperature detected

Predicted Classification
Engine Overheat

Considerations



Example quality matters



Too few examples may reduce reliability



Prompt design is important

Zero-Shot vs Few-Shot



Zero-Shot

No examples in prompt



Few-Shot

A few examples guide the model

BMW Use Cases



Vehicle Diagnostics



Predictive Maintenance



Customer Support



Quality Inspection



Marketing Personalization

Chain-of-thought



Guiding AI to reason step by step for better problem solving.



DEFINITION

Chain-of-thought prompting is a technique where the prompt guides the AI model to break a problem into intermediate reasoning steps before giving the final answer.



BENEFITS



Improves reasoning accuracy
Leads to more reliable and correct answers.



Breaks complex tasks into manageable steps
Simplifies complexity and reduces cognitive load.

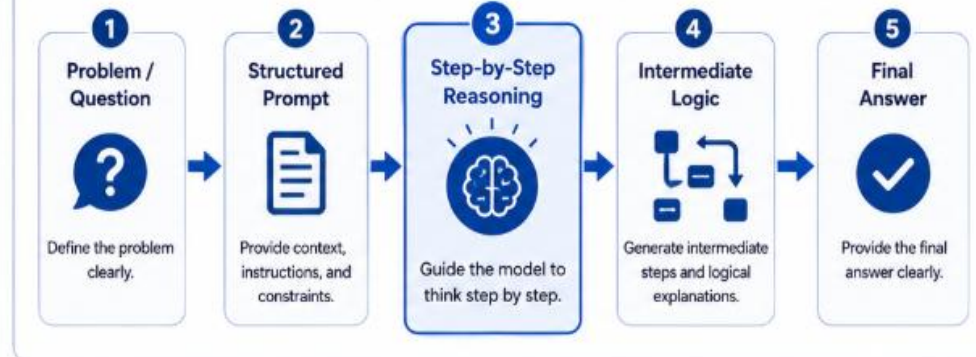


Helps with math, logic, and analysis
Enhances performance on quantitative and analytical tasks.



Produces clearer and more explainable responses
Improves transparency and builds user trust.

HOW IT WORKS



EXAMPLE PROMPT TEMPLATE



“Solve the problem step by step. Show your reasoning clearly, then provide the final answer.”

TYPICAL USE CASES



Math problem solving



Decision analysis



Debugging code



Root-cause analysis



Business reasoning



BEST PRACTICES



Ask the model to reason step by step
Explicitly instruct the model to show its reasoning.



Give clear task context
Provide all relevant details and background information.



Specify the expected output format
Define how the final answer should be structured.



Use for complex or multi-step problems
Best for tasks requiring multiple reasoning steps.



TAKEAWAY:

Chain-of-thought prompting helps AI reason more effectively by making its thinking process structured, sequential, and easier to follow.



Chain-of-thought



CHAIN-OF-THOUGHT PROMPTING IN BMW USE CASES

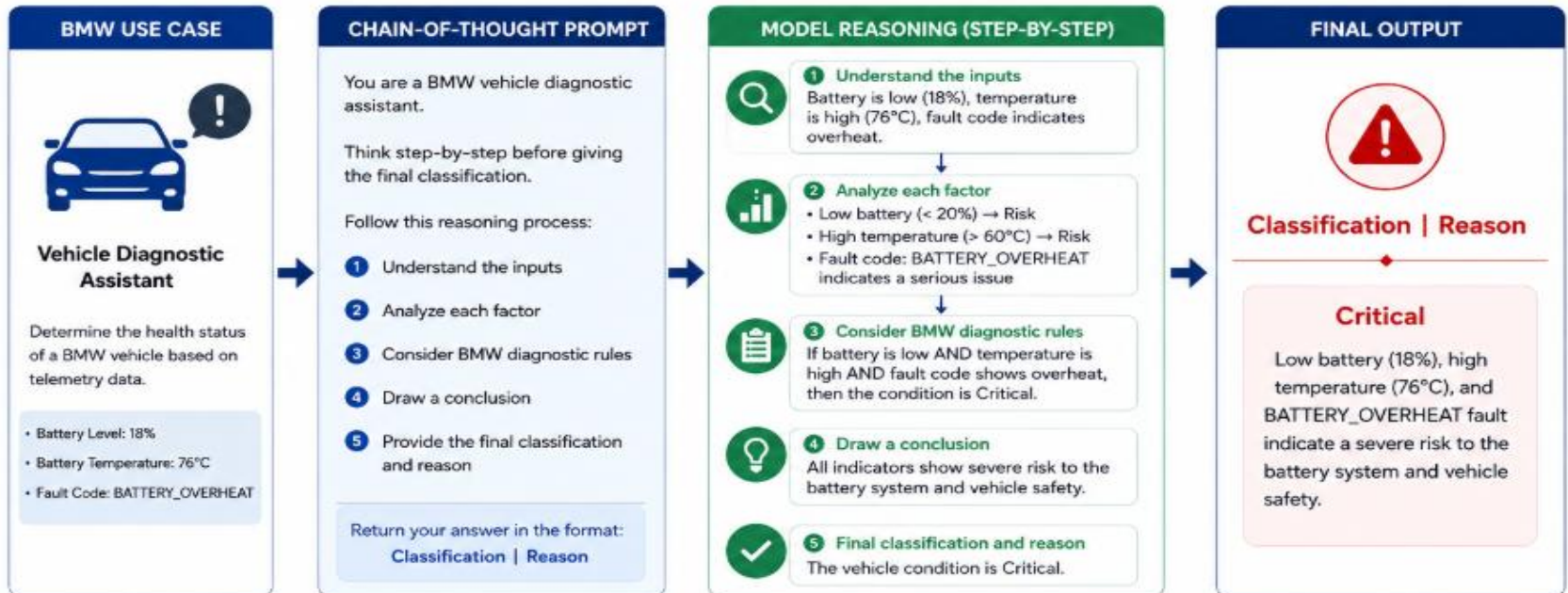


Guide the model to think step-by-step for more accurate and explainable answers



DEFINITION

Chain-of-Thought (CoT) prompting encourages the model to break down a problem into intermediate reasoning steps before giving the final answer.



WHY CHAIN-OF-THOUGHT



Improves accuracy on complex tasks



Makes reasoning transparent



More reliable diagnostics



Better for safety-critical decisions in BMW

Tree of Thoughts (ToT) for better reasoning

Exploring multiple reasoning paths to find the best answer



DEFINITION

Tree of Thoughts prompting allows the AI model to generate multiple thoughts (ideas or reasoning steps) at each stage, explore them like a tree, evaluate, and select the best path to the solution.



KEY BENEFITS



Better Problem Solving

Explores many possibilities leading to more accurate solutions.



Handles Complex Tasks

Decomposes difficult problems into manageable steps.



Reduces Errors

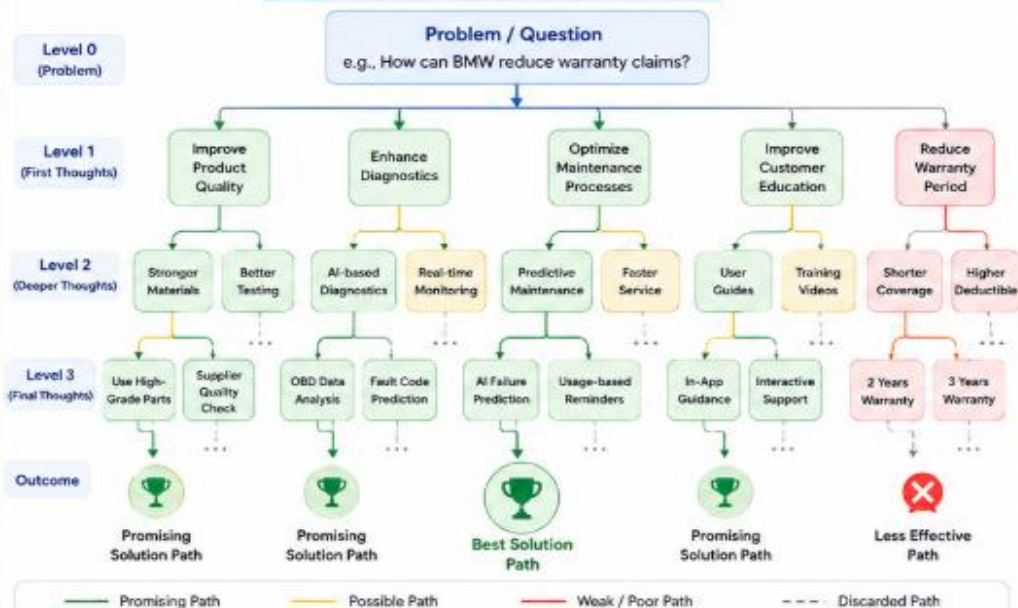
Evaluates different paths and avoids early wrong conclusions.



Improves Reliability

More robust reasoning especially for open-ended and multi-step problems.

HOW TREE OF THOUGHTS WORKS



BEST PRACTICES



Define the goal clearly

State the problem and expected outcome in detail.



Encourage multiple thoughts

Ask the model to generate several possible next steps at each stage.



Evaluate and prune

Score or select the most promising paths and drop weak ones.



Iterate until the best answer

Continue expanding and evaluating until a confident solution is found.



Provide context

Give background information, constraints, and examples when needed.



EXAMPLE PROMPT TEMPLATE

You are solving a complex problem. Use Tree of Thoughts approach.

1. Generate multiple possible next steps.
2. Evaluate each step briefly.
3. Expand the most promising steps.
4. Continue until you find the best solution.
5. Present the final answer and reasoning path.

REAL-WORLD USE CASES



BMW Strategy

Explore multiple strategies to reduce warranty claims.



Product Development

Evaluate different design options and trade-offs.



Supply Chain

Find the best ways to optimize costs & delivery.



Customer Support

Resolve complex customer issues with multiple steps.



Risk Management

Explore risk scenarios and mitigation strategies.



TAKEAWAY

Tree of Thoughts prompting helps AI think like a decision tree—explore many reasoning paths, evaluate options, and choose the best solution with confidence.



Tree of Thoughts (ToT) for better reasoning



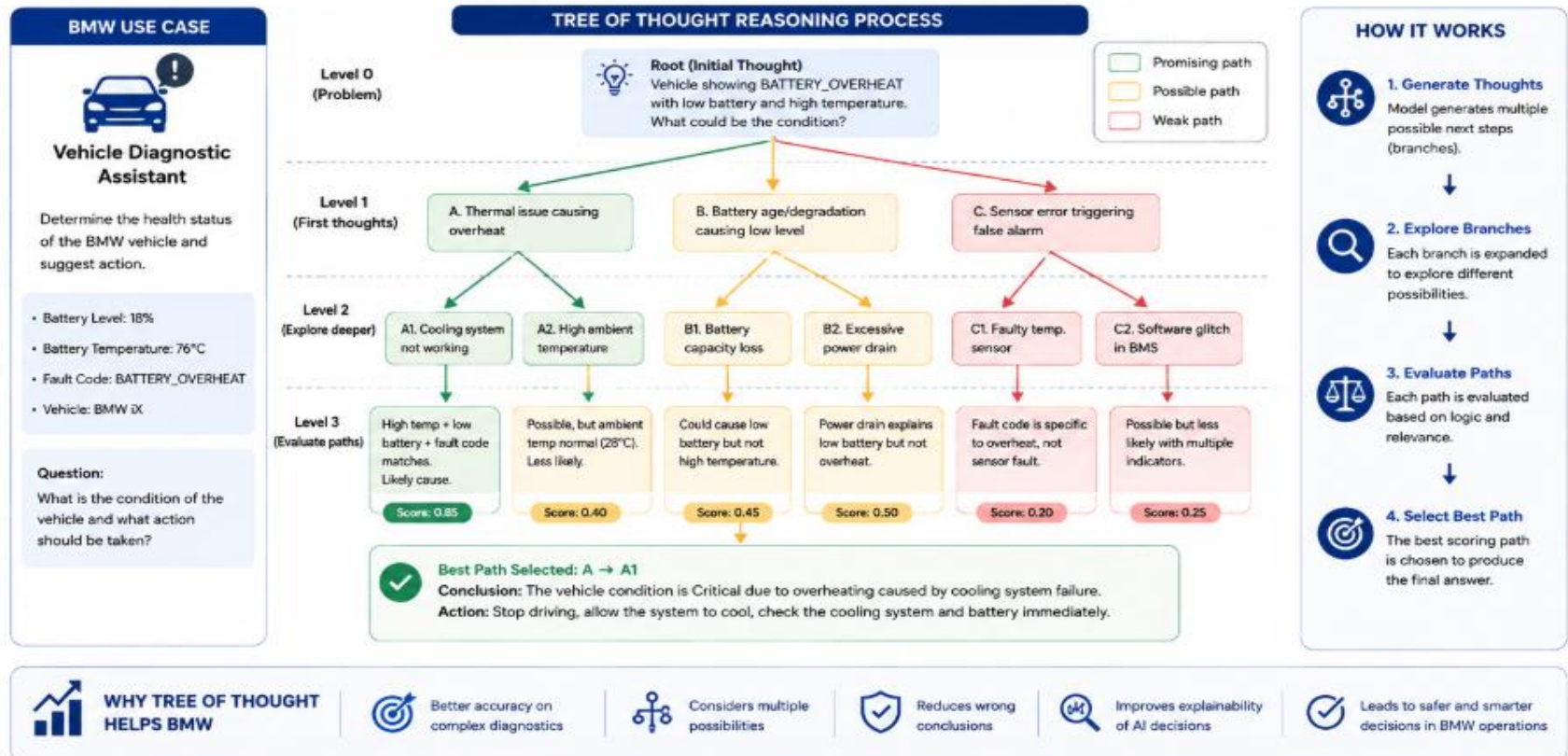
TREE OF THOUGHT (TOT) PROMPTING IN BMW USE CASES

Explore multiple reasoning paths like a tree to find the best answer



WHAT IS TREE OF THOUGHT?

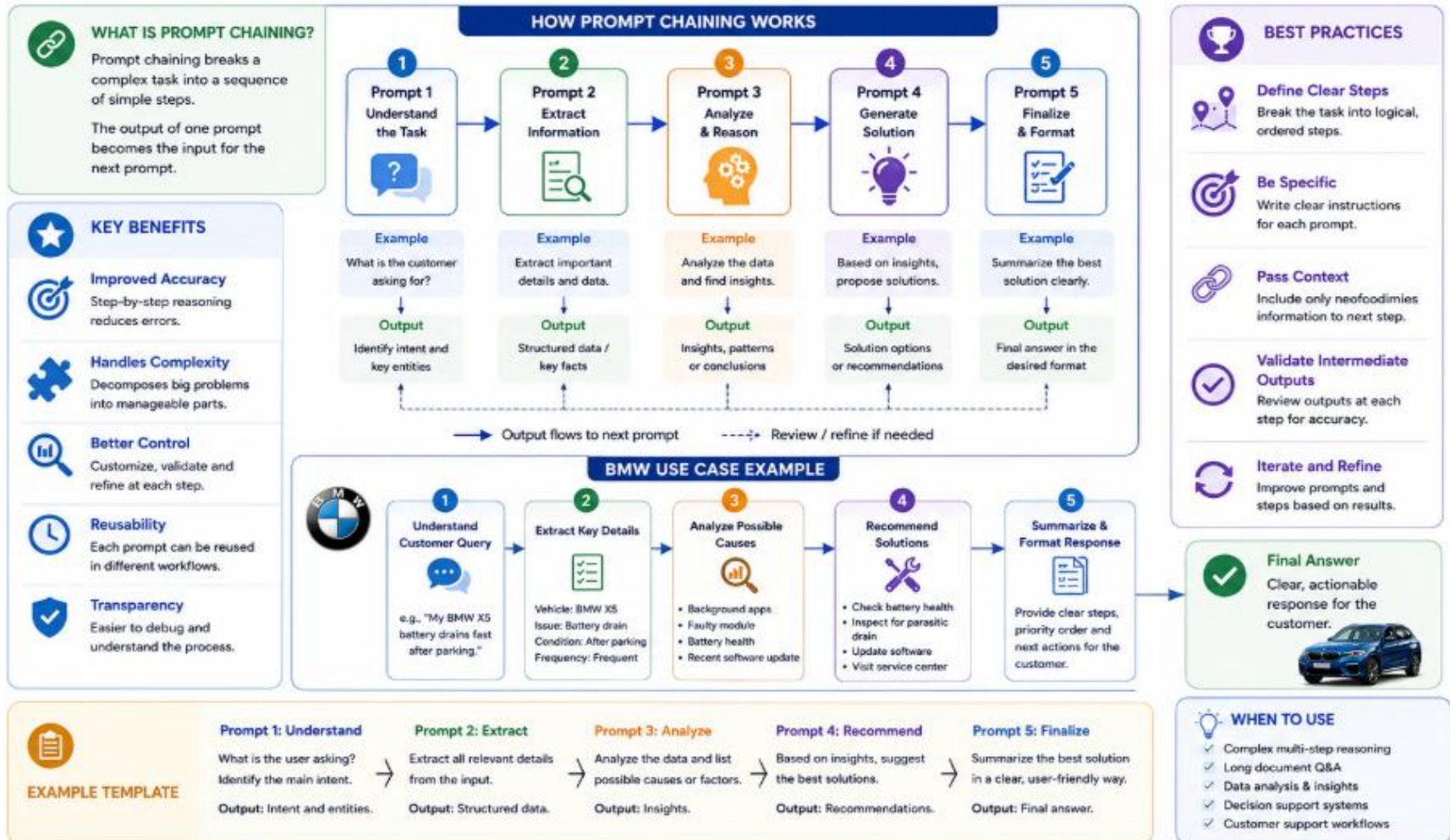
Tree of Thought prompting allows the model to explore multiple reasoning steps (branches) at each step and evaluate different paths to arrive at the best solution.



Prompt Chaining for Complex Tasks



Break complex problems into a series of linked prompts to get accurate, structured results



Prompt Chaining for Complex Tasks



Prompt Chaining in BMW Use Cases

Break a complex BMW problem into a sequence of linked prompts to produce accurate, structured results



What is Prompt Chaining?

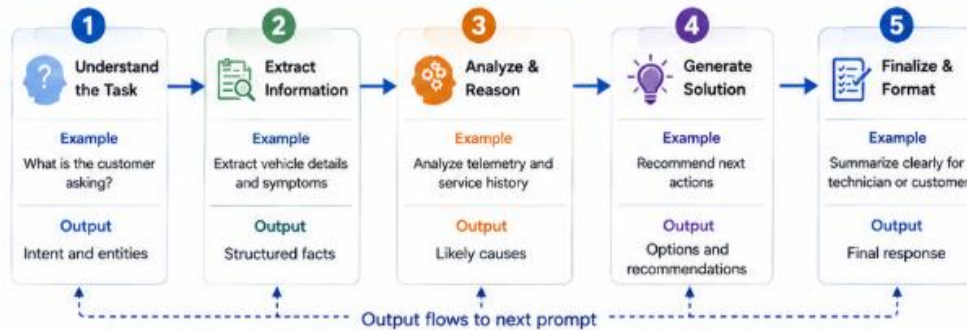
Prompt chaining breaks a complex task into a sequence of smaller prompts.

The output of one prompt becomes the input for the next prompt.

Key Benefits

- Improved accuracy**
More precise and reliable results.
- Handles complexity**
Breaks large problems into manageable steps.
- Better control**
Guide, validate, and steer each step.
- Reusability**
Reuse prompts across different workflows and scenarios.
- Transparency**
Easier to understand and trust the process.

How Prompt Chaining Works



BMW Use Case Example



Example Template



Best Practices

- Define clear steps**
Plan logical, ordered steps.
- Be specific**
Write precise instructions for each prompt.
- Pass context carefully**
Include only what's needed for the next step.
- Validate intermediate outputs**
Check outputs at each step for accuracy.
- Iterate and refine**
Improve prompts and results continuously.

When to Use

- Complex multi-step reasoning
- Long document Q&A
- Data analysis and insights
- Decision support systems
- Customer support workflows



Self-consistency for Multiple Answers

- Self-consistency is a prompting / decoding technique for large language models that improves accuracy (especially on reasoning tasks) by generating multiple answers and selecting the most consistent one.



Self-consistency for Multiple Answers

- **How it works**

- Sample multiple independent responses from the model (usually with temperature > 0 so the answers vary).
- Extract the final answer from each response (e.g., the number, the choice, the conclusion).
- Take a majority vote (or the most frequent answer). That majority answer is treated as the model's final output.
- The intuition is that the correct reasoning path tends to converge on the same answer more often than incorrect paths do. Random errors cancel out; the consistent answer is more likely to be right.



Self-consistency for Multiple Answers

- **Type 1: Classification / discrete questions**
- *"Is BMW's EV strategy aggressive, moderate, or conservative compared to Mercedes and Audi?"*
- Force a parseable label, sample N times, majority-vote:
 - python
 - prompt = `"""Analyze BMW's EV strategy vs Mercedes and Audi.`
- Think step by step, then end with exactly:
- VERDICT: [aggressive | moderate | conservative]"""
- `verdicts = [extract_verdict(llm(prompt, temperature=0.7)) for _ in range(7)]`
- `final = Counter(verdicts).most_common(1)[0][0]`
- If 6/7 samples say "moderate," that's your answer with high confidence. A 4–3 split means the model genuinely finds it ambiguous — worth noting in your output rather than hiding.



Self-consistency for Multiple Answers

- **Type 2: Numeric estimates**
- *"Estimate BMW's global deliveries next year" or "What share of BMW's revenue comes from China?"*
- Sampled numbers won't match exactly, so use the **median** of N samples instead of a vote.
- Optionally report the spread (min–max or interquartile range) as an uncertainty band.
- If the samples span from 1.8M to 2.6M vehicles, that range is honest information, not noise to discard.



Self-consistency for Multiple Answers

- One caution: self-consistency makes the answer *stable*, not *true*.
- If the model's training data is stale, all 10 samples will confidently agree on an outdated figure.
- For factual industry numbers (current sales, market share, latest models), retrieval/search + one good answer beats voting over 10 memorized guesses.
- Self-consistency shines on *reasoning* (comparisons, projections, structured analysis), not fact lookup.

Self-consistency for Multiple Answers

- **Type 3: Multi-factor analysis**
- *"Should a supplier prioritize BMW or Volkswagen as a customer?"*
- Decompose, then vote per dimension:
- Volume outlook → vote: BMW / VW / tie
- Margin & pricing power → vote
- EV transition risk → vote
- Final recommendation → vote
- This shows you *where* consensus breaks. Maybe all samples agree VW wins on volume and BMW on margins, and the 5–2 split only appears on the final weighting — that's a much more useful output than a single opaque answer.



Self-consistency for Multiple Answers

- **Type 4: Open-ended analysis**
- *"What are the biggest risks facing BMW this decade?"*
- Exact voting fails on free text. Two options:
- **Cluster-then-count:** extract the risks each sample names (China exposure, software gap vs Tesla, EU regulations, tariffs...), normalize them, and rank by how many samples mention each. Risks appearing in 8/10 samples are robust; one-off mentions are noise.
- **Universal self-consistency:** paste all N answers back into the model and ask it to produce the answer most consistent with the majority.



Role Prompting - "You are an expert..."

- **Role Prompting – “You Are an Expert...”**
- **Role Prompting** means assigning the AI a specific **persona, profession, or area of expertise** before giving it a task.
- This helps the model respond with the **right perspective, terminology, level of detail, and style.**
- **Simple Formula**
- **Role + Context + Task + Constraints + Output Format**



Role Prompting - "You are an expert..."

- **BMW Example**
- You are a **BMW vehicle diagnostic expert**.
Analyze the following telemetry:
 - Battery Level: 18%
 - Battery Temperature: 76°C
 - Fault Code: BATTERY_OVERHEAT
- Identify:
 - Vehicle condition
 - Main issue
 - Possible cause
 - Recommended action
- Return:
Condition | Issue | Cause | Action



Role Prompting - "You are an expert..."

- **Expected Output**
- **Critical | Battery overheating | Possible cooling-system issue | Stop safely and inspect the battery cooling system**

Role Prompting

Role	Example Task
BMW Diagnostic Expert	Analyze vehicle faults
BMW Data Analyst	Analyze sales or telemetry trends
BMW Service Advisor	Explain maintenance recommendations
BMW Manufacturing Engineer	Investigate production defects
BMW Software Architect	Design connected-vehicle architecture
BMW Security Expert	Review API and vehicle-system security
BMW Customer Support Agent	Answer customer vehicle questions



Why Role Prompting Helps

- Provides the model with a **clear perspective**
- Improves **domain-specific responses**
- Controls **tone and terminology**
- Produces more **relevant and structured answers**
- Useful for creating specialized **AI assistants and agents**
- **Key takeaway:**
Instead of asking “**Analyze this problem**”, use “**You are a BMW vehicle diagnostic expert. Analyze this problem...**”



Instruction Following

- **Instruction following** means giving the model clear rules about **what to do, what not to do, and how to format the response**.
- **BMW Example**
- You are a BMW diagnostic assistant.
Analyze only the supplied telemetry.
Do not assume missing information.
Classify the condition as Normal, Warning, or Critical.
Return only:
Classification | Reason
- **Why it helps:** improves consistency, control, and output formatting.



Avoiding Hallucinations

- **Hallucination** happens when an AI generates information that sounds correct but is **unsupported, incorrect, or invented**.
- **BMW Prompt to Reduce Hallucination**
- Answer only using the supplied BMW telemetry and service information.
Do not invent fault codes, specifications, or causes.
If the available information is insufficient, respond:
“Insufficient information to determine.”



Avoiding Hallucinations

- **Useful Techniques**

- Provide trusted context or documents
- Tell the model to use only supplied data
- Ask it to identify uncertainty
- Require citations when using documents
- Use RAG for enterprise knowledge
- Validate critical outputs before action

- **Key idea:**

Ground the model in trusted data instead of allowing unsupported assumptions.



Iterative Prompt Refinement

- **Iterative prompt refinement** means improving a prompt step-by-step until the model produces the required result.
- **Version 1 – Too General**
 - Analyze this BMW vehicle.
- **Version 2 – Better**
 - Analyze the battery level, temperature, and fault code.



Iterative Prompt Refinement

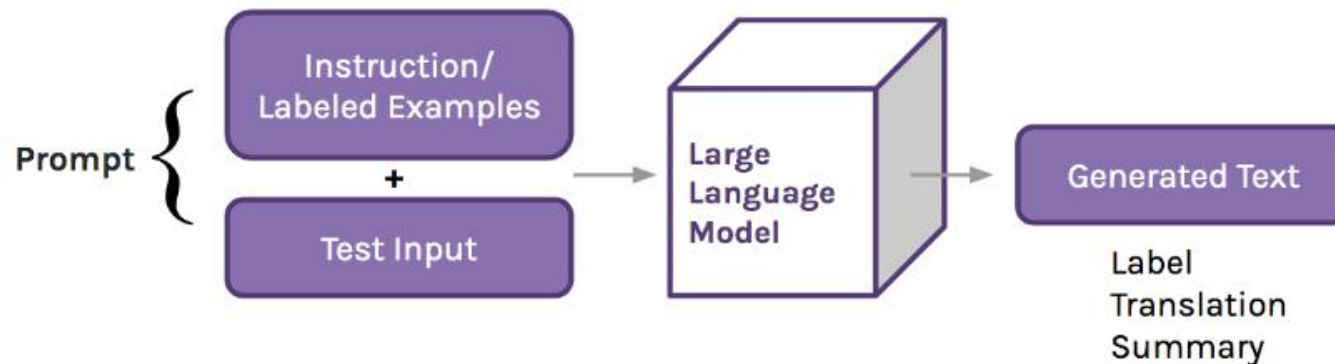
- **Version 3 – Refined**
- You are a BMW vehicle diagnostic expert. Analyze the following telemetry:
 - Battery Level: 18%
 - Battery Temperature: 76°C
 - Fault Code: BATTERY_OVERHEAT
- Use only the supplied data. Classify the condition as Normal, Warning, or Critical. Return:
Classification | Issue | Recommended Action

Refinement Cycle

- **Write Prompt → Test → Review Output → Identify Problem → Improve Prompt → Retest**
- **BMW Example of Refinement**
- **Problem:** Output is too verbose
→ Add: “Return no more than 3 lines.”
- **Problem:** Model invents causes
→ Add: “Do not infer causes not supported by the data.”
- **Problem:** Output varies
→ Add a fixed output format.

LLM

- A **Large Language Model (LLM)** is an advanced AI that processes a **prompt**, which includes instructions and input data, to generate human-like text.
- It is used for tasks like text generation, translation, and summarization, based on its understanding of patterns in language.

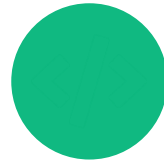


LLM



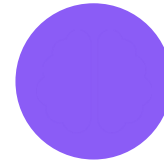
LARGE

Billions to trillions of parameters
learned from text



LANGUAGE

Works with text: reading, writing,
translating, summarising



MODEL

A mathematical system, not a
database or search engine



From Millions to Trillions: The Scale of LLMs

2017

- Transformer architecture invented (Google, "Attention is All You Need")

2018

- BERT released — 110M parameters, bi-directional understanding

2020

- GPT-3 — 175B parameters; demonstrated emergent reasoning

2022

- ChatGPT launched — 100M users in 60 days (fastest app in history)

2024+

- Multimodal models: text, image, audio, video & code combined



Modern LLMs are trained on 10–15 trillion words — roughly 10,000× all text ever written before 2000.

Components of LLM

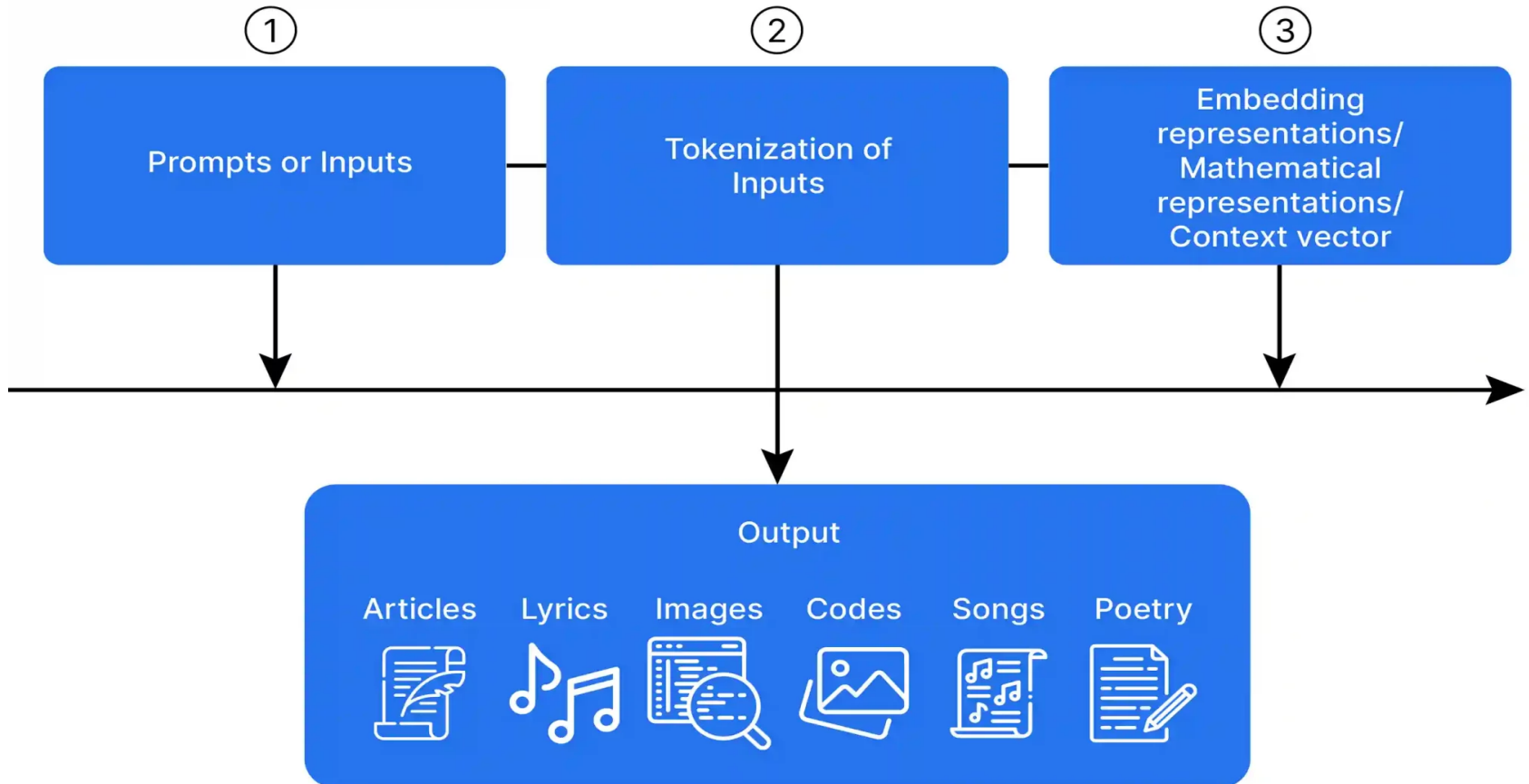


Working of LLM



- **Prompts or Inputs:** The process begins with feeding a prompt or input to the system, such as a question, statement, or request.
- **Tokenization of Inputs:** The input is broken down into smaller units or tokens (words, phrases, or characters) to make them understandable to the machine.
- **Embedding Representations:** The tokens are converted into mathematical representations that capture their meaning and context, allowing the model to process them effectively.
- **Output:** Based on the processed information, the LLM generates outputs such as articles, lyrics, images, code, songs, or poetry.

How LLMs Work

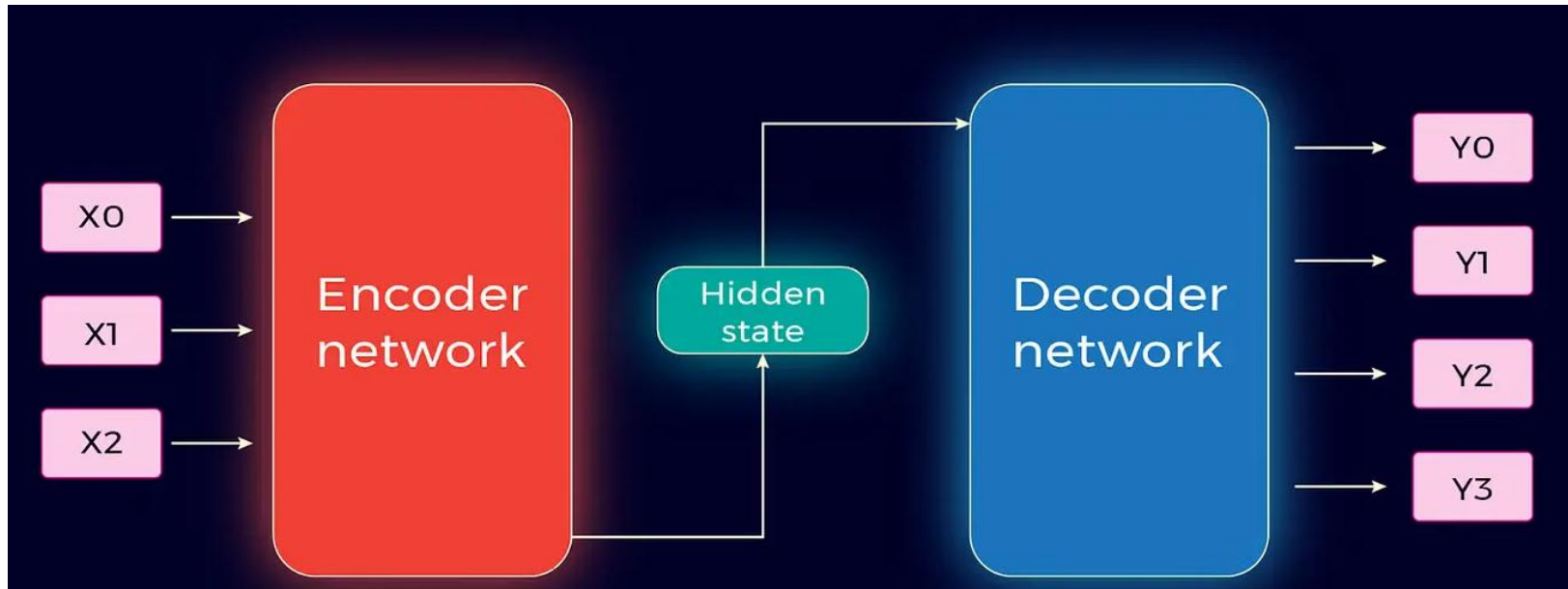


Use Case



Title	LLM in Summarization
 Objective	To condense lengthy documents or articles into concise, coherent summaries.
 Simple Explanation	The ability to generate a compact version of text that retains essential information
 Core AI Technologies	Large Language Models
 Real-World Applications	News Summarization: Creates shorter news summaries that cover the main events. Scientific Research: Breaks down complex research papers into digestible summaries.
 Learning Highlights	Excels at distilling important facts and themes from extensive text.
 Key Benefits	• Provides time-efficient solutions for content-heavy industries. • Improves accessibility to information through brief yet comprehensive summaries.

Architecture of LLM





Architecture of LLM

- The architecture shown is for a **Large Language Model (LLM)**, which uses an encoder-decoder structure:
- **Encoder Network:** Processes input data (e.g., text tokens like X_0 , X_1 , X_2) and converts it into a dense representation, capturing the essential information and context.
- **Hidden State:** This dense representation is stored as a context vector and passed between the encoder and decoder.
- **Decoder Network:** Uses the hidden state to generate the corresponding output (e.g., Y_0 , Y_1 , Y_2 , Y_3). It generates text, predictions, or sequences based on the input data.

Use Cases of LLM



LLMs: Revolutionizing Key Sectors



- **ChatGPT and GPT-4**, since their introduction, have revolutionized industries with tailored, domain-specific applications.



Example of LLM



A transformer is used in language modeling to predict the next word in a series based on the previous context.

Input: "The cat sat on the" **Output:** "mat"

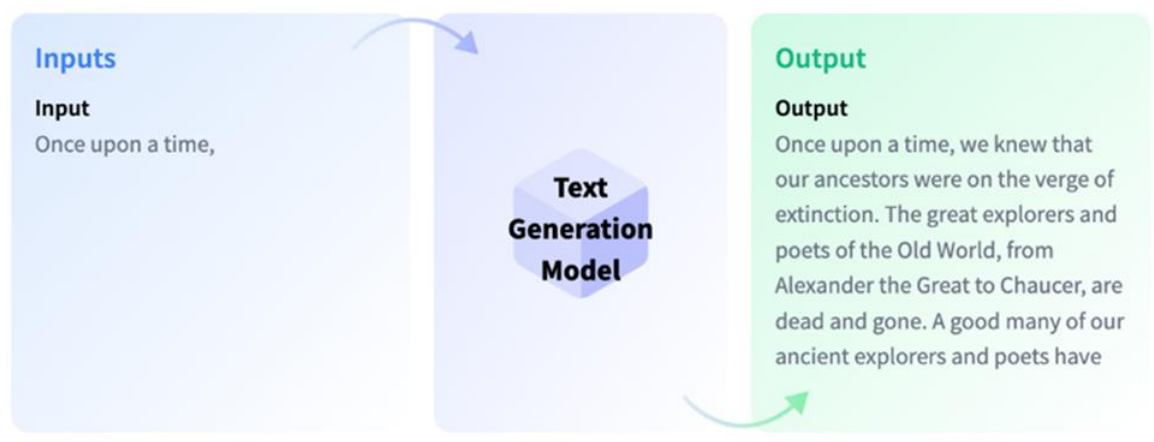
The transformer predicts "mat" from "The cat sat on the" using self-attention mechanisms to analyze the links between all words in the sequence.

LLMs like GPT can capture long-range text dependencies using the transformer design, resulting in high-quality text production and interpretation.

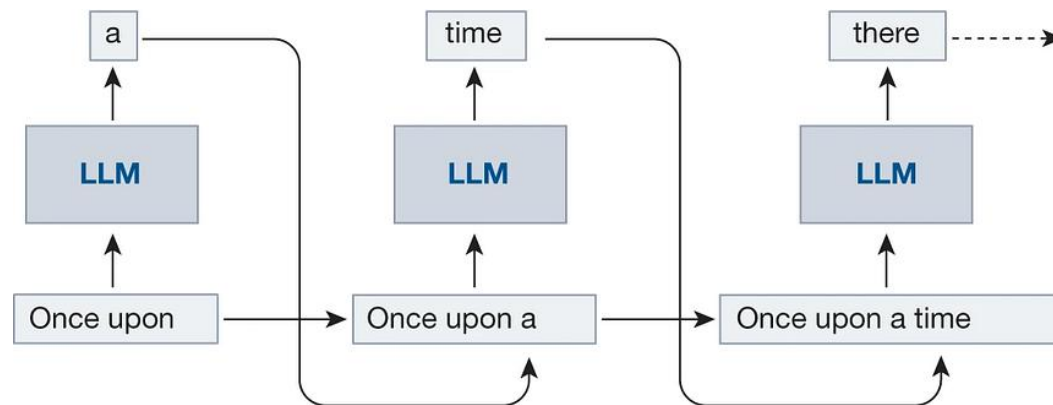
- Text generation automatically generates human-like text from a prompt or context.
- LLMs like GPT-3, trained on massive text data, can generate coherent and contextually appropriate text.

How Text Generation Works?

- A sentence, phrase, or word is used as an input prompt to generate text. This prompt starts text generation.



- The image demonstrates how a Language Model (LLM) generates text. It starts with an input word like "a", then progressively predicts the next words ("Once upon", "time", "there") by using each output as input for the next step, creating coherent sentences.

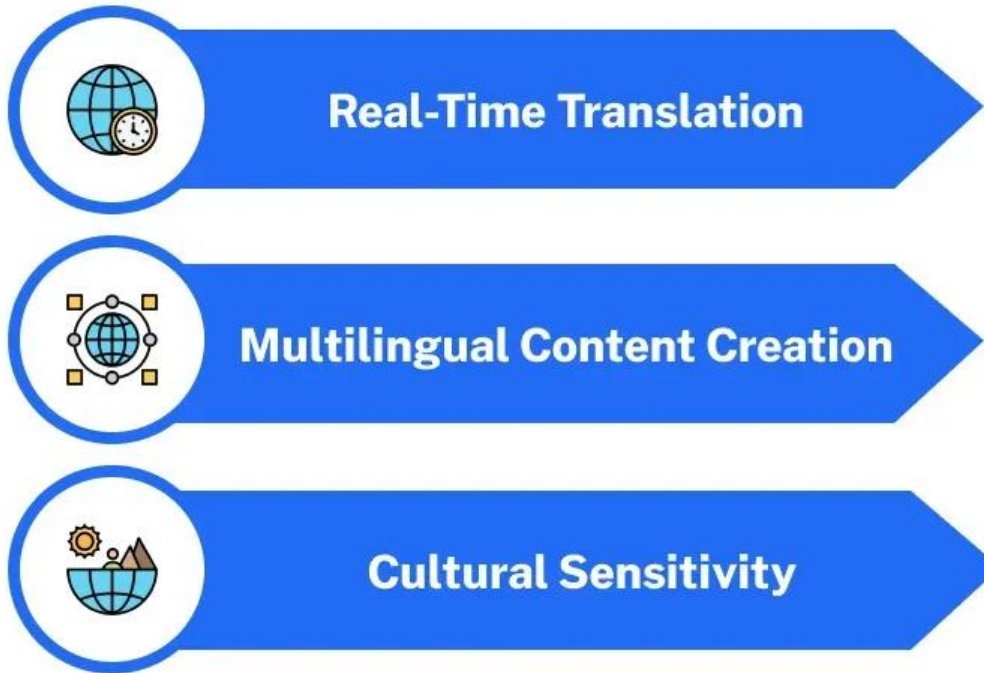


Enhancing Global Communication with LLMs



- LLMs (Large Language Models) play a crucial role in enhancing global communication by offering real-time translations, creating multilingual content, and ensuring cultural sensitivity.
- They help businesses expand their reach, improve engagement, and navigate diverse markets effectively.

Enhancing Global Communication with LLMs





Text Translation

- The translation preserves meaning and context while changing languages. LLMs and other algorithms fuel machine translation systems, enabling cross-language communication and information access.
- **Source Text (English):** "The cat sat on the mat."
- **Translated Text (French):** "*Le chat s'est assis sur le tapis.*"

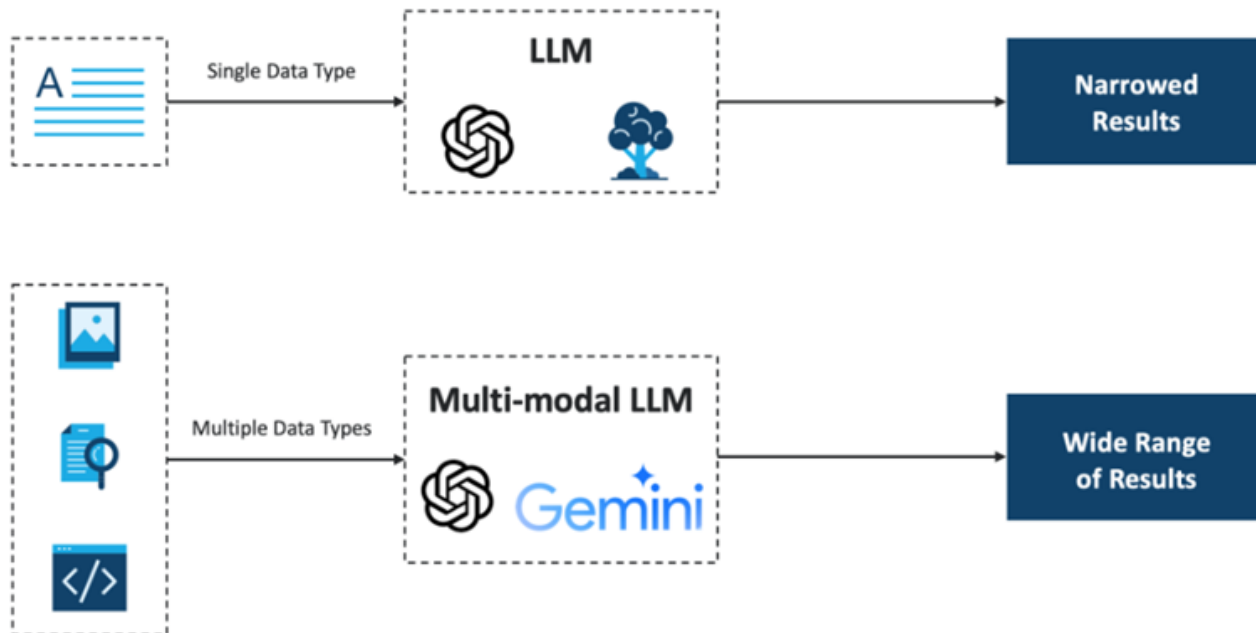


Creative Text Formats

- **Lyric Generation:** LLMs can generate lyrics for various genres, such as pop, rock, hip-hop, and ballads. They can write memorable hooks, lyrics, and choruses that tell stories or evoke emotions.
- **Fictional Character Dialogue Generation:** LLMs can explore interpersonal relationships, tensions, and resolves through dialogue generation. Plays, movies, video games, and interactive storytelling can use written dialogue.
- **Scriptwriting:** LLMs can help with scriptwriting for visual media such as films, TV, and theatre. They can generate scene descriptions, character dialogue, and plot developments based on predefined themes or prompts.
- **Experimental Text:** LLMs can generate experimental or avant-garde text formats that push the boundaries of traditional language use. This includes text-based art, concrete poetry, surrealist prose, and linguistic experiments.

Multimodal Large Language Models

- **LLM (Single Data Type):** Focuses on one type of input (like text) and provides narrowed results.
- **Multi-modal LLM:** Processes multiple types of data (text, images, code) and produces a wide range of results.



Question Answering and Knowledge Extraction



Semantic Search

LLMs can understand the nuances of user queries and document content, enabling more accurate retrieval of relevant documents. LLMs can match query semantics to document content instead of keyword matching.



Query Expansion

LLMs can automatically expand user queries by generating additional relevant terms or phrases, improving recall in information retrieval systems.



Personalized Search

LLMs can leverage user context, preferences, and behavior to personalize search results, providing more relevant and tailored recommendations.



Multimodal Retrieval

LLMs can incorporate information from multiple modalities, such as text, images, and audio, to provide more comprehensive search results. A search engine might employ LLMs to understand text searches and image or video content.



Domain-specific Retrieval

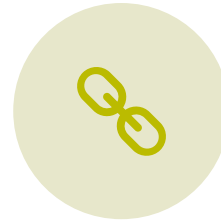
LLMs can be fine-tuned on domain-specific data to optimize retrieval performance for specific domains, such as legal documents, medical literature, or scientific research papers.

Knowledge Base Construction with LLMs



Entity and Relation Extraction:

LLMs can extract entities (e.g., person names, organization names) and relationships (e.g., works-for, located-in) from unstructured text data, enabling the construction of knowledge graphs or databases.



Named Entity Linking: LLMs can link extracted entity mentions to entries in existing knowledge bases, such as Wikipedia or Freebase, enriching the knowledge base with additional information and context.



Fact Extraction and Verification:

LLMs can extract factual information from text and verify its accuracy against trusted sources, helping to build reliable knowledge bases



Automatic Summarization: LLMs can summarize large volumes of text data, distilling key information into concise summaries that can be used to populate knowledge bases more efficiently.



Cross-lingual Knowledge Base Construction:

LLMs trained on multilingual data can facilitate the construction of knowledge bases in multiple languages, enabling cross-lingual information access and knowledge sharing.



Top Models for Developers (2026)

Use Case	Recommended Model
General coding, architecture, debugging	OpenAI GPT Models (GPT-5 family)
Large codebase understanding	Anthropic Claude (Claude Sonnet family)
IDE autocomplete	GitHub Copilot
Enterprise Azure ecosystem	Azure AI Foundry models including OpenAI models
Self-hosted/open-source	Meta AI Llama family
Cost-sensitive coding assistants	Mistral AI models



Azure OpenAI vs OpenAI

Feature	OpenAI	Azure OpenAI
Provider	OpenAI	Microsoft Azure
Models	GPT, image, speech models from OpenAI	Same OpenAI models hosted through Azure
Authentication	OpenAI API Key	Azure Resource + Azure Key/Entra ID
Billing	Directly from OpenAI	Through Azure subscription
Enterprise Security	Good	Strong Azure enterprise controls
Data Residency	Limited options	Azure region selection
Integration	Simple	Deep Azure integration
SLA & Governance	Standard	Enterprise-grade Azure governance
Best For	Startups, developers, learning	Large enterprises, regulated industries



LLM vs HuggingFace

LLM	Hugging Face
A Large Language Model	A platform and ecosystem for AI models
Understands and generates language	Helps you find, download, run, fine-tune, and deploy models
Examples: GPT, Llama, Mistral, Qwen, Gemma	Examples: Hugging Face Hub, transformers, datasets, pipeline()
It is the AI model itself	It is the tooling/platform around models
Can be accessed via API or run locally	Can host and run many different LLMs

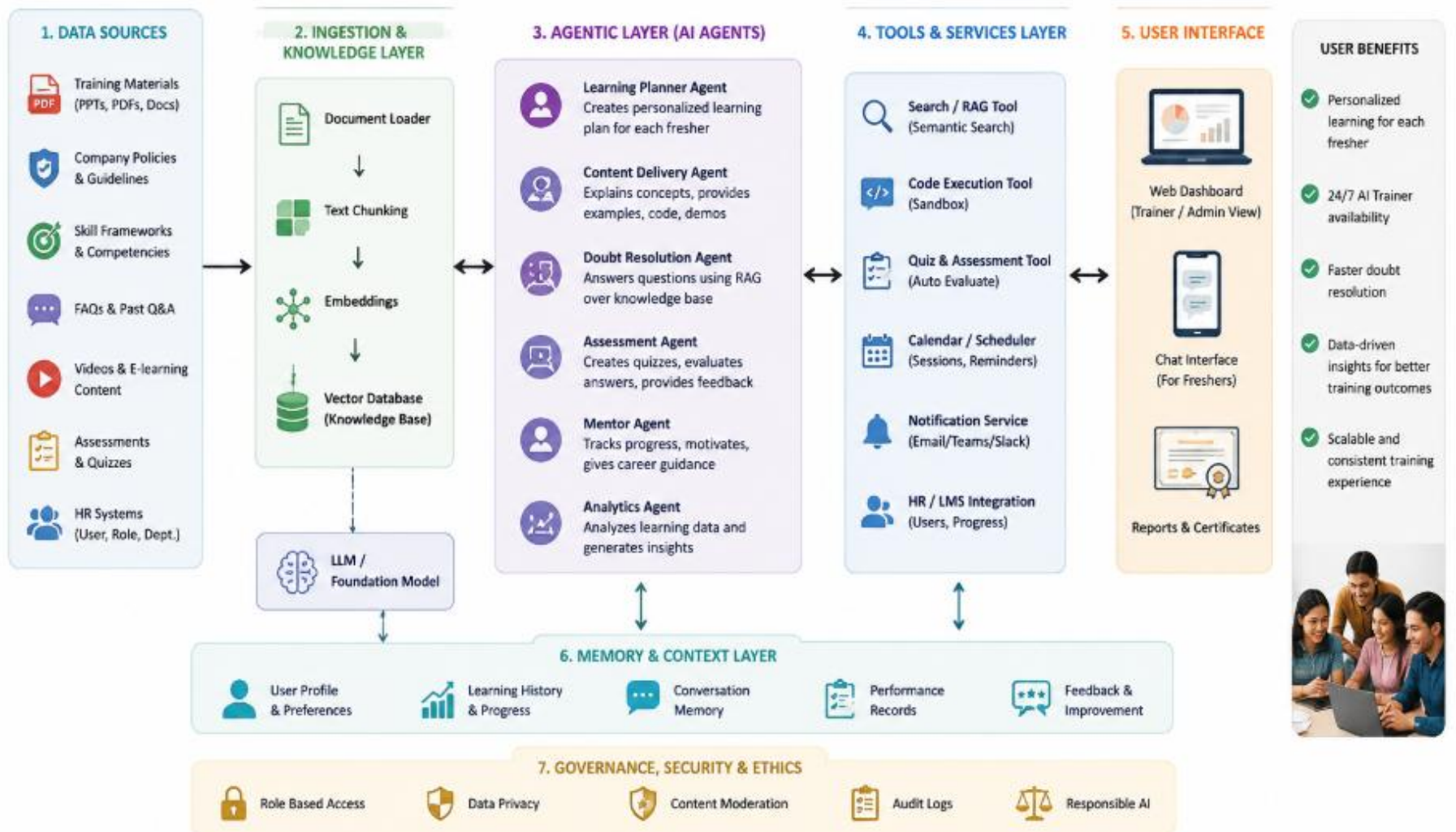
LLM vs HuggingFace

Need	Better choice
Fastest setup	Hosted LLM API
Best conversational answers	Hosted LLM API
Strong reasoning	Hosted LLM API
Run locally without API	Hugging Face
Open-source models	Hugging Face
More model choice	Hugging Face
Fine-tuning/customization	Hugging Face
Zero-shot classification with scores	Hugging Face
Simple prompt → answer demos	Hosted LLM API
Privacy/offline lab	Hugging Face

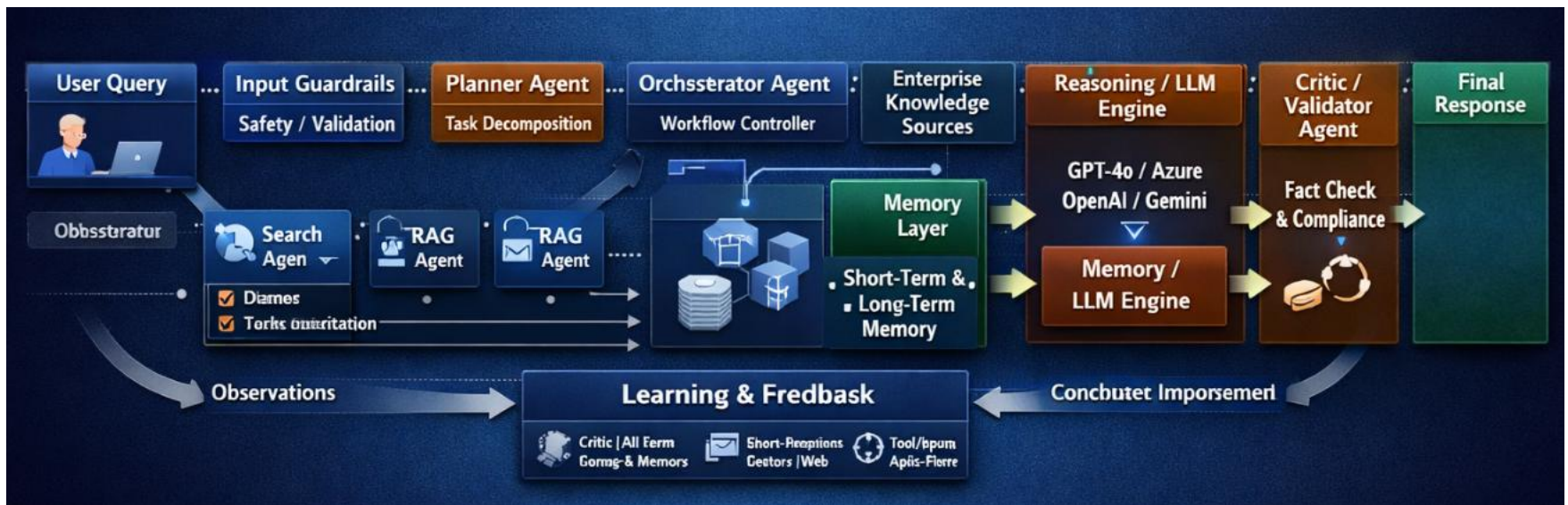


- **1. What is an AI Agent?**
- An **AI agent** is a software system that uses an AI model to understand a goal, decide what to do, use available tools, observe the result, and continue until it reaches a stopping condition.

Agentic AI



Agentic AI



Agentic AI



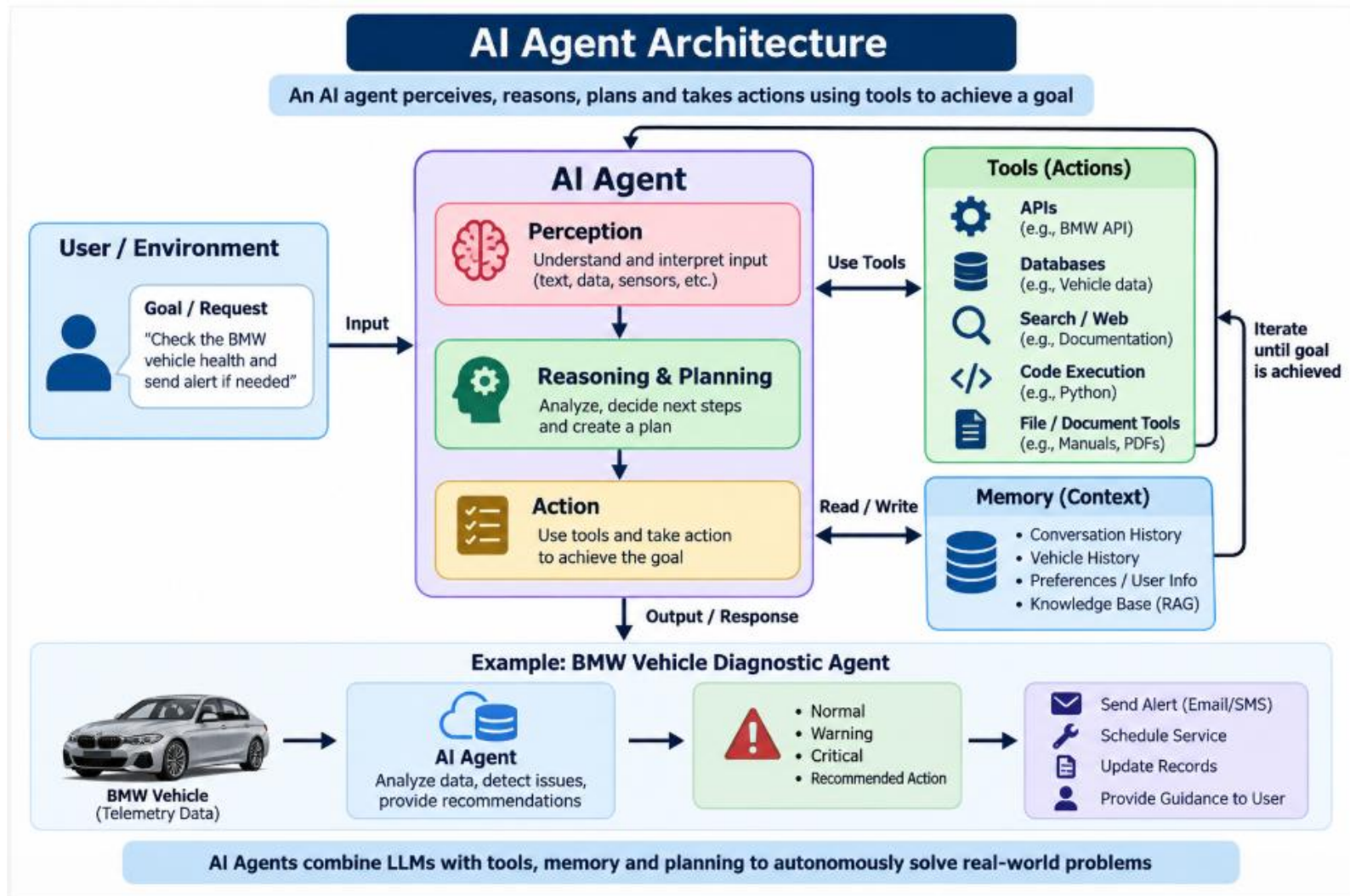
How an autonomous agent turns a goal into action through an iterative reason-act loop



Agentic AI



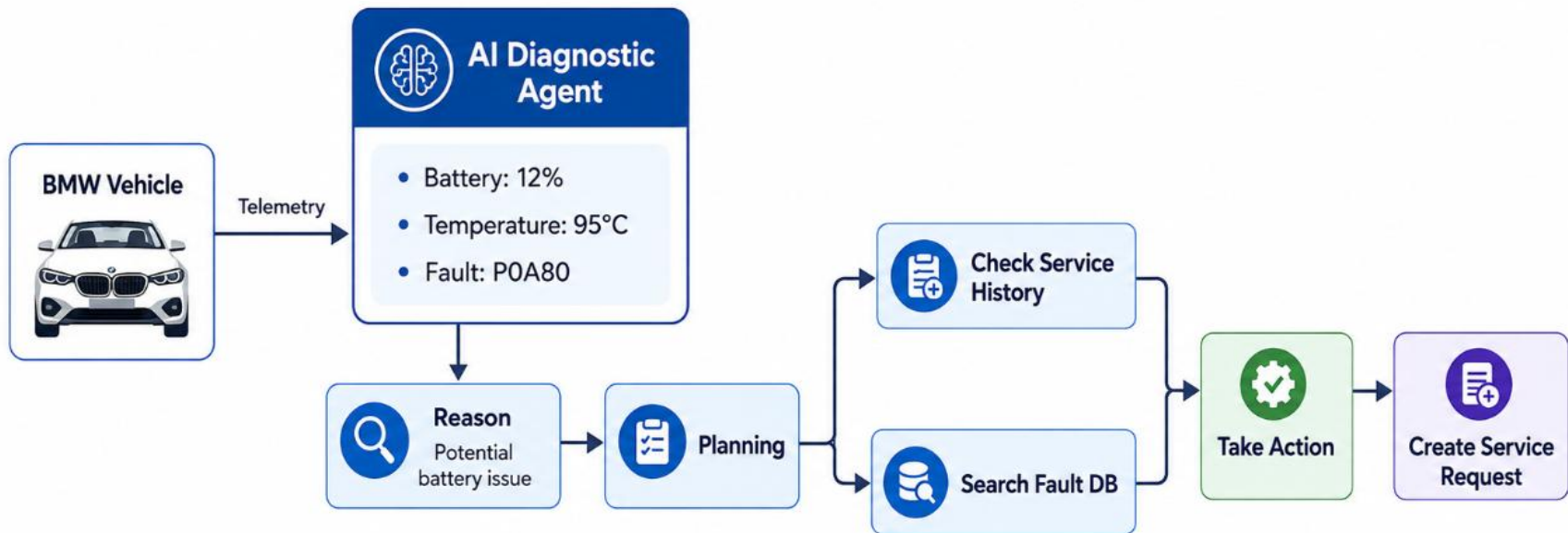
Agents



Perceive → Reason → Act

Stage	Meaning	BMW example
Perceive	Receive information from environment	Receive BMW telemetry
Reason	Analyze information and decide what it means	Battery = 12%, temperature = 95°C
Act	Perform an operation	Send alert / create service ticket

Perceive → Reason → Act

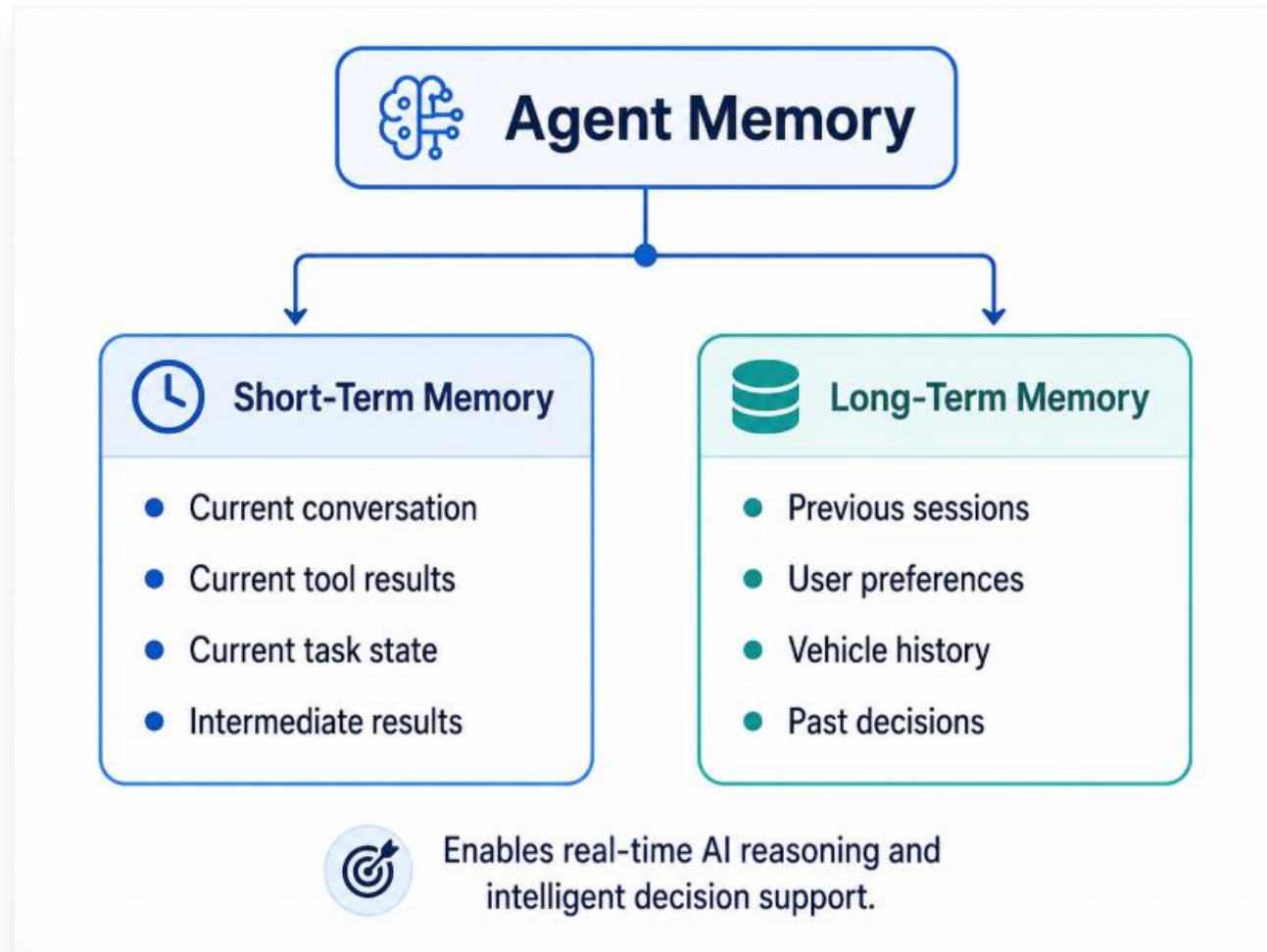




Main Components of an Agent

- A production agent normally combines:
 - **LLM / reasoning model:** understands the request and decides what action to take.
 - **Tools:** functions, APIs, databases, search systems, calculators, Python, enterprise services, etc.
 - **Memory/state:** retains conversation context, intermediate state, user preferences, or information from previous executions.
 - **Planning:** breaks a large goal into smaller tasks and determines their order.
 - **Reasoning/decision making:** selects the next appropriate action based on the current state and tool results.
 - **Observation:** reads the output returned by a tool.
 - **Control loop:** decides whether another action is necessary or the task is complete.

Memory



Memory

- For example:
 - Vehicle ID: BMW1001
 - Model: BMW i4
 - Previous Issue: Battery overheating
 - Last Service: 15-Aug-2026
- When another temperature problem appears, the agent can incorporate that historical context.
- LangGraph specifically supports persistent state and both short-term working memory and longer-term memory patterns for stateful agents.

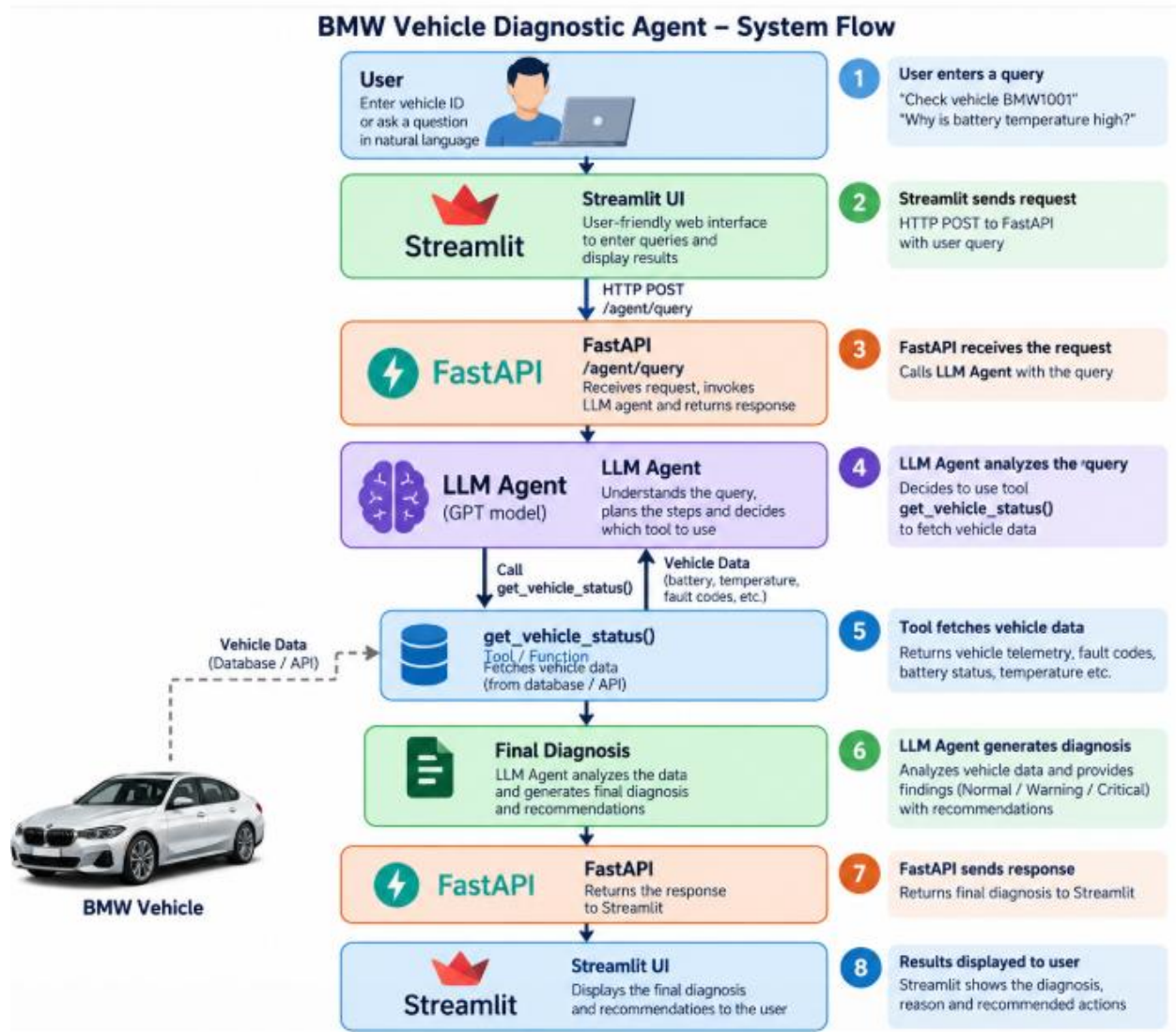
Reasoning vs Planning

Concept	Question
Reasoning	What does this information mean and what should I do next?
Planning	What sequence of tasks should I perform to reach the goal?
Tool use	Which external capability should I invoke?
Memory	What information should I retain?
Observation	What happened after my action?

Reasoning vs Planning

- Example:
- Battery = 12%
- Temperature = 95°C
- |
- ▼
- Reasoning
- "Battery condition may be critical"
- |
- ▼
- Planning
 - 1. Check fault code
 - 2. Check service history
 - 3. Determine severity
 - 4. Create service request

Agent Steps

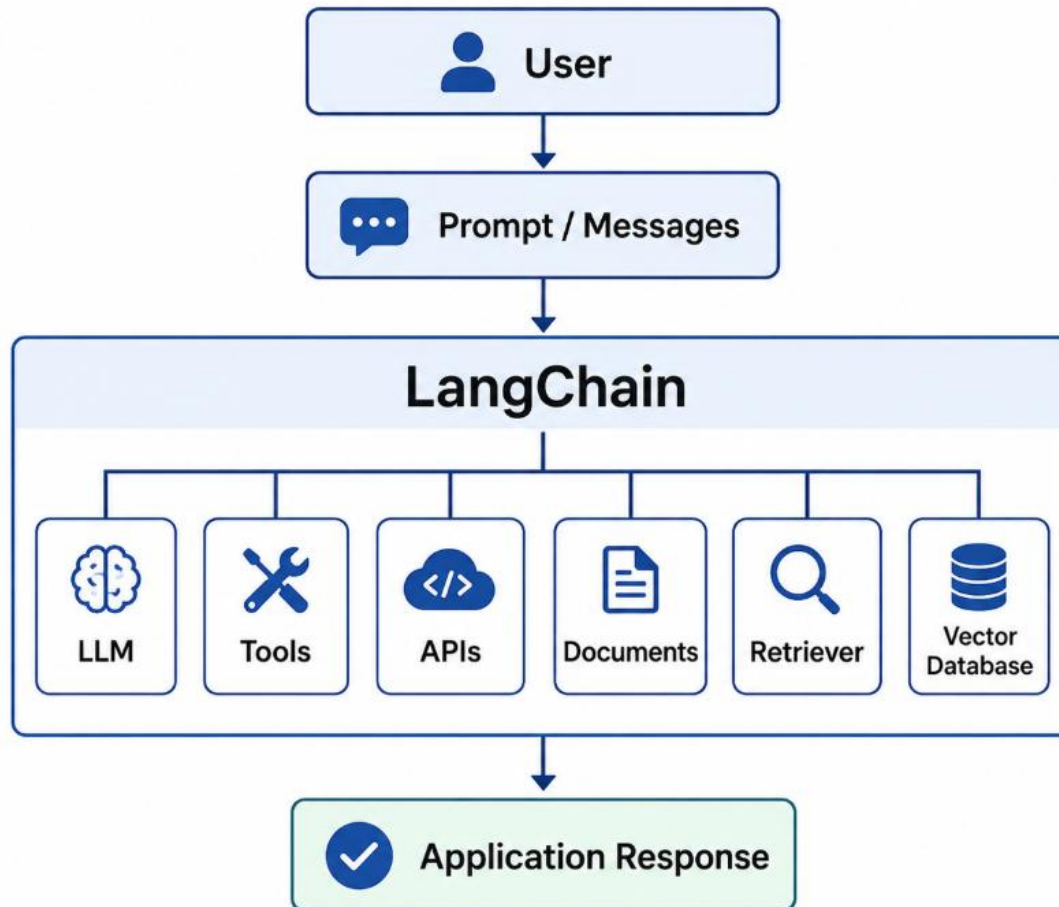


Agent Frameworks

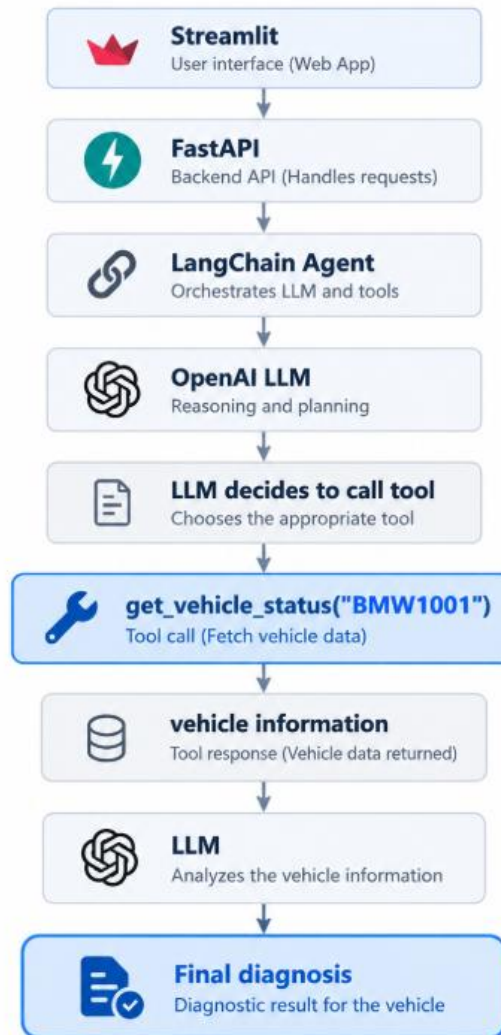


Framework	Main idea	Best suited for
LangChain Agents	High-level model + tools agent loop	Getting agents running quickly
LangGraph	Graph/state-machine orchestration	Complex production workflows
AutoGPT	Platform for building and running autonomous agent workflows	Goal-driven automation

- **LangChain** is a framework for building applications around Large Language Models.
- It helps connect an LLM with **prompts, tools, external data, retrievers, APIs, and agent workflows** through a common application layer.
- Current LangChain emphasizes a standard model interface and configurable agents, with LangChain agents built on top of LangGraph.



LangChain





Why use LangChain in an API

- **Prompt management** – reusable prompt templates with variables.
- **LLM integration** – switch between OpenAI, Azure OpenAI, Gemini, local models.
- **Structured output** – return predictable JSON-like results.
- **RAG integration** – retrieve information from BMW manuals or knowledge bases before calling the LLM.
- **Tool calling** – let the model call telemetry, service-history, or diagnostic APIs.
- **Workflow chaining** – connect multiple steps such as retrieve → analyze → recommend.
- **Agents** – allow the LLM to decide which tool/API to use.
- **Less custom glue code** – standard components for models, retrievers, tools, and workflows.



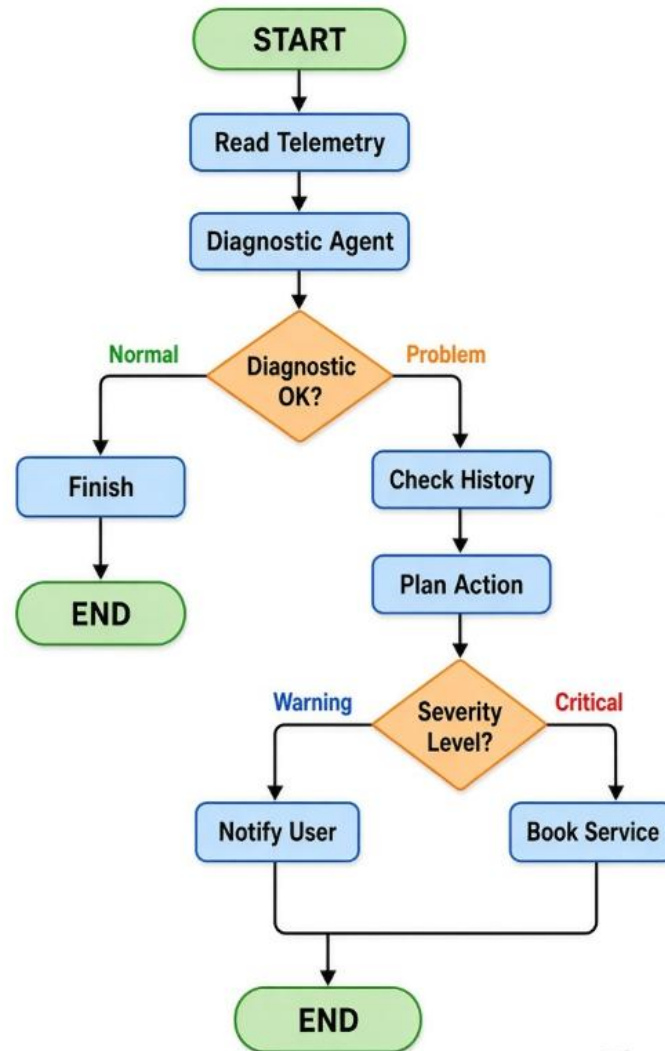
Main LangChain Concepts

Component	Purpose	BMW Example
Chat Model	Connect to an LLM	OpenAI / Azure / Gemini
Prompt Templates	Reusable dynamic prompts	Insert vehicle ID and telemetry
Structured Output	Return predictable data	Severity, Cause, Action
Document Loaders	Load external documents	BMW service manuals
Text Splitters	Break documents into chunks	Split technical manuals
Embeddings	Convert text into vectors	Embed service documents
Vector Store	Store and search embeddings	FAISS / Chroma / Qdrant
Retriever	Find relevant information	Retrieve repair instructions
Tools	Connect external functions/APIs	Vehicle telemetry API
Agents	Decide which tools to use	Vehicle diagnostic assistant



Langgraph

- **LangGraph** is a framework for building **stateful, multi-step LLM workflows and agents using graphs**.
- Think of it as:
 - Nodes = Tasks
 - Edges = Flow between tasks
 - State = Data shared between tasks
 - Graph = Complete workflow
- LangGraph is designed for more complex workflows where you need branching, loops, memory, human approval, retries, or multiple agents.
- It can be used independently of LangChain, although LangChain agents themselves use LangGraph underneath.

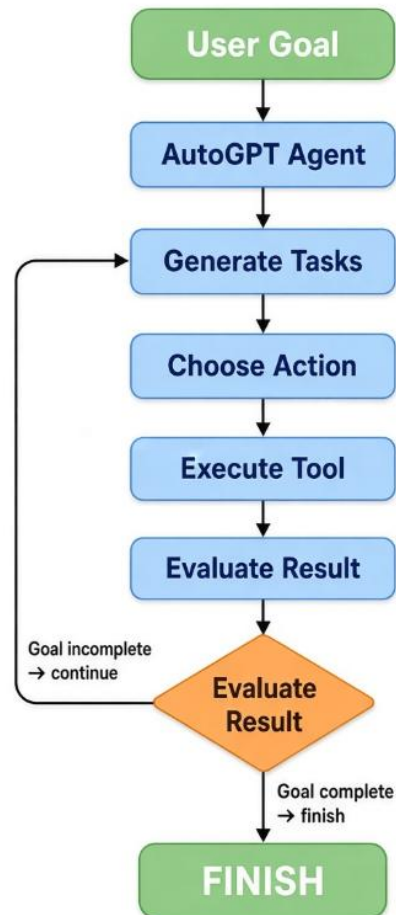




Langchain vs Langgraph

LangChain	LangGraph
Easier LLM application building	Complex workflow orchestration
Prompt → LLM → Output	Graph of multiple steps
Chains, RAG, tools	Nodes, edges, state
Good for simpler flows	Good for branching and looping
Faster to start	More control
Basic agent development	Advanced/stateful agents

- AutoGPT focuses more heavily on goal-driven agent automation.





- The current AutoGPT project describes itself as an open-source platform for building, deploying, and running agents that execute complete workflows, including on-demand, scheduled and trigger-based runs.

- **AutoGPT agents automate multi-step workflows from a goal or trigger.**
- Examples:
 - Vehicle Diagnostic Agent
 - Predictive Maintenance Agent
 - Service Booking Agent
 - Roadside Assistance Agent
 - Customer Support Agent
 - Parts Inventory Agent

Agent	Trigger	Action	Output
Vehicle Diagnostic	Live telemetry	Check battery, temperature, fault code	Warning / service request
Predictive Maintenance	Sensor threshold	Analyze history and failure risk	Maintenance recommendation
Service Booking	Critical issue	Find center, check slots, book service	Confirmed appointment
Roadside Assistance	Breakdown event	Get location, identify issue, dispatch help	Assistance request

Agent	Trigger	Action	Output
Parts Inventory	Low stock	Check parts, suppliers, reorder	Replenishment request
Warranty Claim	Fault + service request	Validate warranty and history	Warranty claim
Manufacturing Quality	Defect detected	Analyze batch and past defects	Quality alert
Fleet Management	Fleet telemetry	Identify risky vehicles	Fleet health report

Agent	Trigger	Action	Output
Customer Support	Customer message	Search KB, recommend fix, escalate	Resolved case
Sales Lead	Website lead	Classify interest, suggest model, update CRM	Qualified lead
Market Intelligence	Schedule	Track competitors and market trends	Weekly report
Incident Management	API alert	Check logs, classify severity, create ticket	Incident workflow

Lab Practice-1



Title	Exploring LLM Capabilities Through Open-Source Implementations
Problem Statement	Large Language Models (LLMs) have demonstrated impressive capabilities in various NLP tasks such as text generation, summarization, translation, and code completion. However, understanding their potential and limitations requires hands-on experimentation. This tutorial will guide you through working with an open-source LLM (like Hugging Face's transformers models) to explore its capabilities, including text generation, summarization, and question-answering. – Demo Done

Lab Practice-2



Title	Hands-on: Generating Different Text Styles with LLMs
Problem Statement	Large Language Models (LLMs) can generate text in various styles, including formal, casual, poetic, technical, and humorous tones. Understanding how LLMs adapt to different writing styles can be valuable for content creation, marketing, storytelling, and automation. In this hands-on exercise, we will experiment with an open-source LLM to generate text in different styles by providing specific prompts. – Demo Done

Lab Practice-3

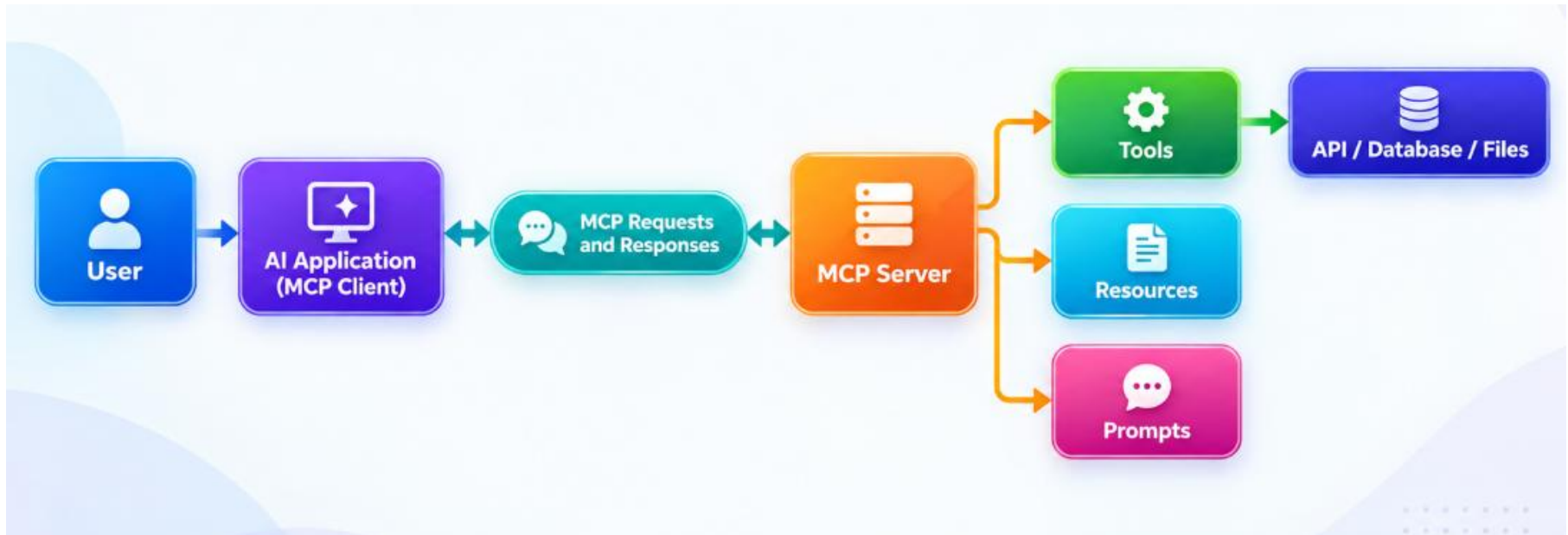


Title	Hands-on: Building a Knowledge Base or Question-Answering System Using LLMs
Problem Statement	Organizations and researchers often need efficient ways to extract, store, and retrieve knowledge from large text corpora. Large Language Models (LLMs) can be leveraged to build a knowledge base or a question-answering (QA) system by extracting key information from text and answering user queries accurately. In this hands-on exercise, we will create a simple QA system using an open-source LLM, demonstrating its ability to process and retrieve relevant information.- Demo Done

- **MCP (Model Context Protocol)** is a standard way for AI applications and agents to connect to **external tools, data, prompts, and services** without writing a different custom integration for every application.
- It is often described as a “USB-C port for AI”: one common interface so any compatible host can talk to any compatible server without custom one-off integrations.

- **MCP (Model Context Protocol)** is a standard way for AI applications and agents to connect to **external tools, data, prompts, and services** without writing a different custom integration for every application.
- It is often described as a “USB-C port for AI”: one common interface so any compatible host can talk to any compatible server without custom one-off integrations.

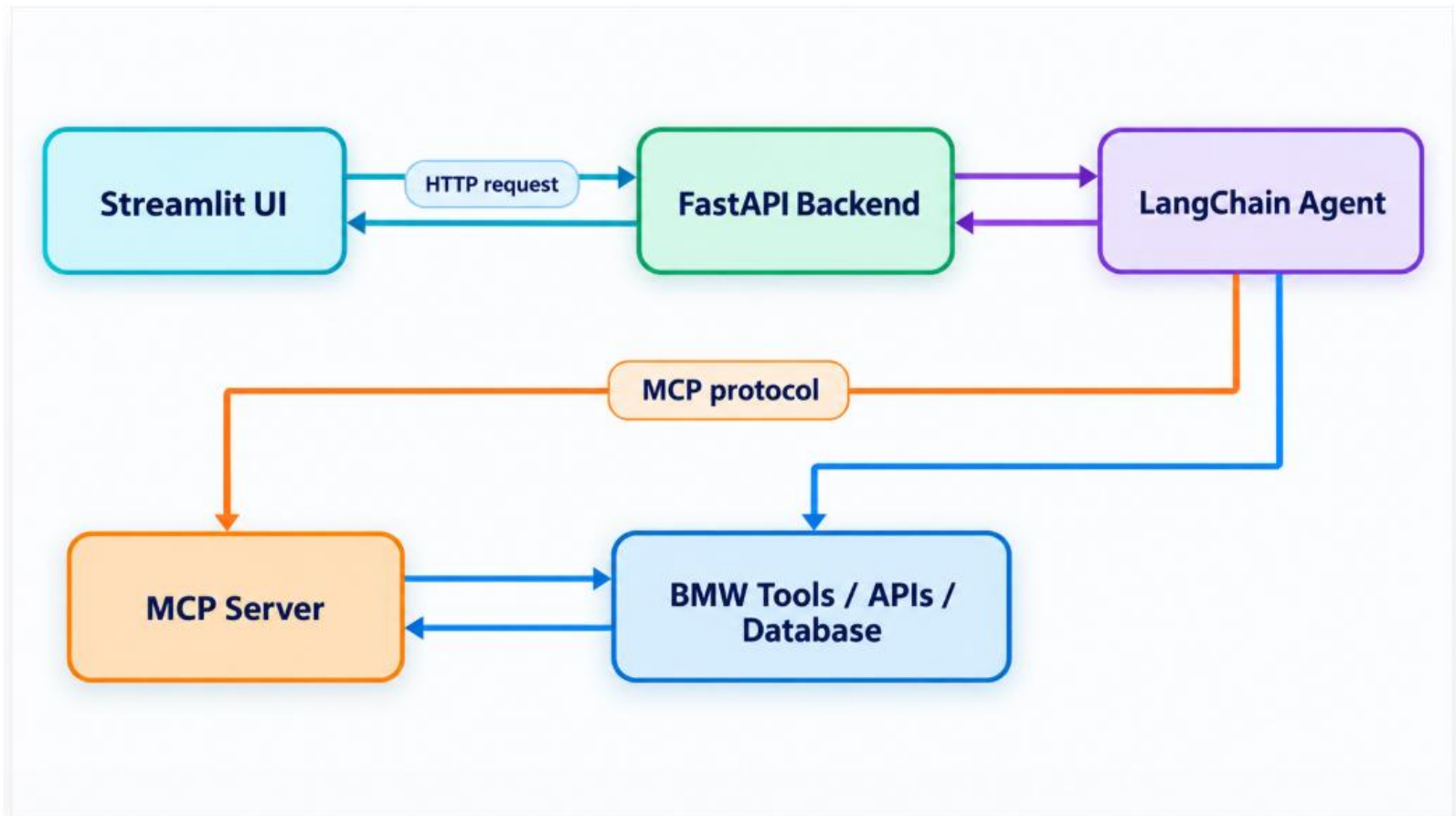
Core Architecture



Main capabilities

Capability	Purpose	BMW example
Tools	Functions the AI can execute	<code>get_vehicle_status("BMW1001")</code>
Resources	Read-only contextual data	Vehicle manuals, telemetry records
Prompts	Reusable prompt templates	Vehicle diagnostic instructions
Transport	Communication mechanism	Local stdio or remote HTTP

Main Capabilities



Main capabilities

Component	Responsibility
Streamlit	Accepts the user's vehicle question and displays the result
FastAPI	Provides the application REST API
LangChain agent	Uses the LLM to decide which tool to call
MCP client	Connects the agent to the MCP server
MCP server	Publishes tools using the Model Context Protocol
BMW tool	Retrieves vehicle information from an API or database

MCP components

- **MCP host**
- The host is the complete AI application.
- In this project:
 - FastAPI application + Agent = MCP Host
- The host is responsible for:
 - Connecting to MCP servers
 - Managing the LLM
 - Enforcing security and approvals
 - Controlling the conversation
 - Passing relevant context to the model



MCP components

- **MCP client**
- The client maintains the connection between the host and one MCP server.
- It can:
 - Discover tools
 - List resources
 - Retrieve prompts
 - Invoke tools
 - Receive tool results
- One application can have multiple clients:

Agent

- |— Vehicle MCP client
- |— Service-booking MCP client
- |— Inventory MCP client



MCP server

- The server exposes selected business capabilities:

BMW MCP Server

- └─ Tool: `get_vehicle_status`
 - └─ Tool: `create_service_request`
 - └─ Resource: vehicle fault-code guide
 - └─ Prompt: `diagnose_vehicle`
- The server does not normally decide the complete conversation.
 - It makes capabilities available to the host and model.

Transport layer

- The transport carries JSON-RPC messages between client and server.

Transport	Suitable for
stdio	Local server started as a subprocess
Streamable HTTP	Remote, web-hosted or production server
HTTP + SSE	Older compatibility scenarios

Tools

- A tool is a function the model can choose to invoke.
- Tools can query APIs, perform calculations or produce side effects.
- Every tool has a unique name and a schema.

```
Tool name: get_vehicle_status
```

```
Input:
```

```
{  
  "vehicle_id": "BMW1001"  
}
```

```
Output:
```

```
{  
  "battery_level": 12,  
  "temperature": 92,  
  "fault_code": "P0A80"  
}
```



Tool schema

- A schema tells the model:
 - Tool name
 - Tool purpose
 - Required arguments
 - Argument types
 - Expected result

Tool schema

```
{  
  "name": "get_vehicle_status",  
  "description": "Get current diagnostic information for a BMW vehicle",  
  "inputSchema": {  
    "type": "object",  
    "properties": {  
      "vehicle_id": {  
        "type": "string",  
        "description": "BMW vehicle identifier such as BMW1001"  
      }  
    },  
    "required": ["vehicle_id"]  
  }  
}
```



Embeddings

- Numerical representations of text, images, or other data in a high-dimensional space.
- Similar concepts are placed close together, enabling semantic search.
- Example: "The mitochondria is the powerhouse of the cell" → 384-dimensional vector.
- Example:
 - Laptop
- becomes:
 - [0.12, -0.45, 0.87, ...]
- Question:
 - Notebook Computer
- becomes:
 - [0.13, -0.44, 0.86, ...]
- Notice:
 - Laptop $\approx$ Notebook Computer

Real-World Example

- Documents:
 - Employees receive 20 annual leaves.
- Question:
 - How many leaves do employees get?
- Keyword search may fail.
- Embedding search finds:
 - leave $\approx$ annual leaves
- because meaning is similar.



Embeddings: meaning as coordinates

An embedding model maps a piece of text to a list of numbers — a vector, often 384 to 3072 dimensions long. Text with similar meaning lands in nearby regions of that space.



Dense & continuous

Every dimension carries a slice of meaning — not a keyword flag.



Semantic, not lexical

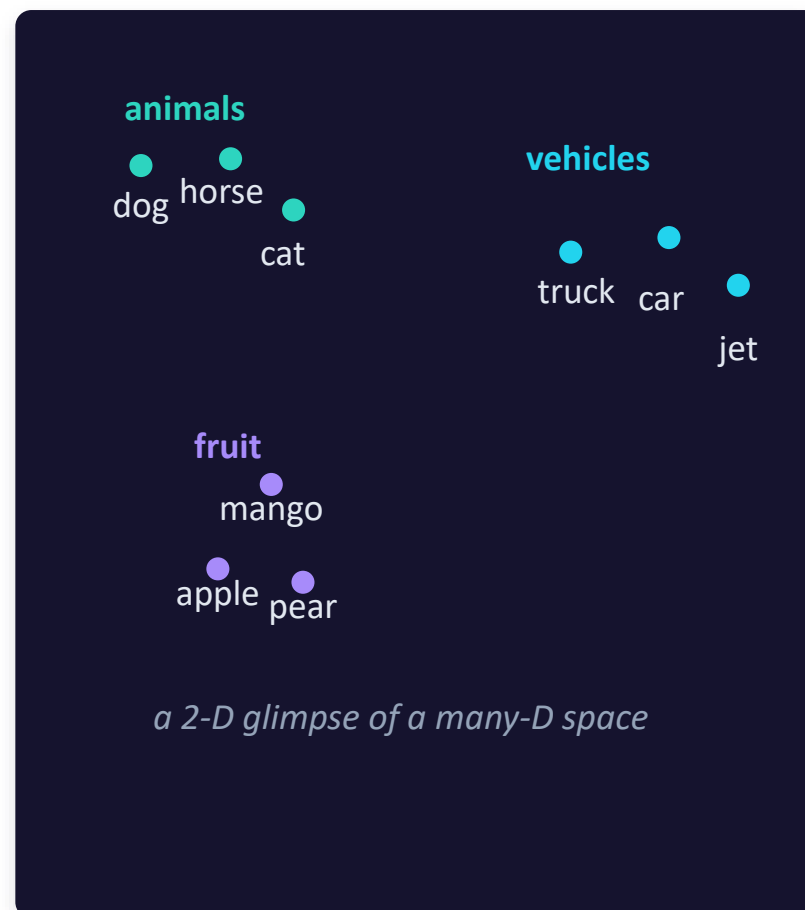
“car” and “automobile” sit close even with zero shared letters.



Fixed length

Any input — a word or a page — becomes one same-sized vector.

Hands on Completed



Measuring how close two vectors are

Retrieval is just a nearest-neighbor search: embed the query, then find the stored vectors pointing in the most similar direction.

Cosine similarity

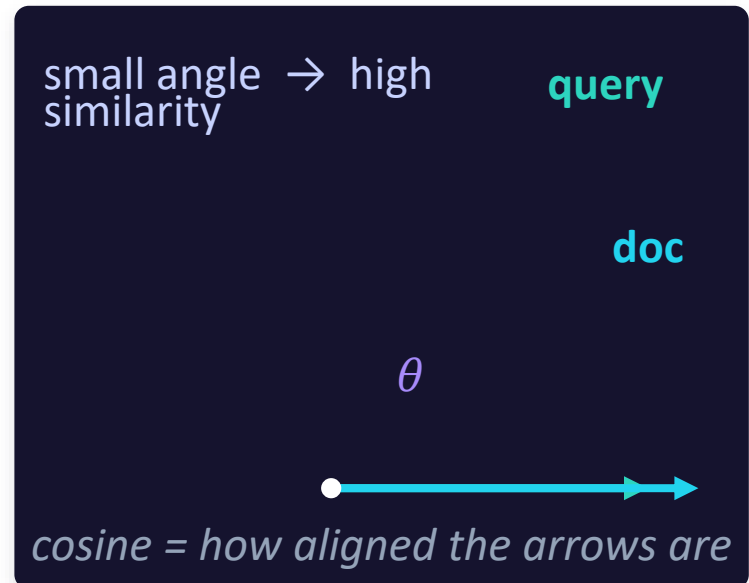
Compares the angle between vectors, ignoring length. The default for text.

Dot product

Angle and magnitude together — common when vectors aren't normalized.

Euclidean (L2)

Straight-line distance between the two points in space.



Approximate Nearest Neighbor (ANN) indexes make this search fast at scale — more on the next slide.



What Does Dimension Mean?

- Embedding:
 - "Laptop battery is not charging"
- becomes:
 - [0.12, -0.45, 0.88,]
- The number of values = embedding dimension.
- Example:
 - 384 numbers
- means:
 - 384-dimensional vector

What Does Dimension Mean?

- **Strategy 1: Small Dimensions (128–384)**
- Examples:
 - all-MiniLM-L6-v2 (384)
- Advantages:
 - ☒ Fast retrieval
 - ☒ Less memory
 - ☒ Cheap storage
 - ☒ Faster indexing
- Suitable for:
 - Chatbots
 - FAQ Systems
 - Small RAG Projects
 - HR Assistants

What Does Dimension Mean?

- **Strategy 1: Small Dimensions (128–384)**
- **Example**
 - 100,000 documents
 - 384 dimensions
- **Storage:**
 - $100000 \times 384 \times 4$ bytes
 ≈ 153 MB
- **Very efficient.**

What Does Dimension Mean?

- **Strategy 2: Medium Dimensions (768)**
- Examples:
 - BERT Base
 - MPNet
- Advantages:
 -  Better semantic understanding
 -  Better retrieval quality
- Disadvantages:
 -  More memory
 -  Slightly slower search
- Suitable for:
 - Enterprise Search
 - Knowledge Management
 - Document Retrieval

Embedding Dimension Strategies

How many dimensions should my embeddings have?

Model	Dimensions
all-MiniLM-L6-v2	384
BERT Base	768
OpenAI text-embedding-3-small	1536
OpenAI text-embedding-3-large	3072
MPNet	768



Vector Databases

- Vector databases are specialized systems designed to store, index, and retrieve high-dimensional embeddings efficiently.
- They are the backbone of semantic search and Retrieval-Augmented Generation (RAG), enabling LLMs to ground responses in external knowledge.



Vector Databases

- **Vector Databases**
 - Specialized systems for storing and querying embeddings.
- Use **Approximate Nearest Neighbor (ANN)** search to find semantically similar items quickly.
- Popular options: **FAISS (Facebook AI)**, **Qdrant (Rust-based, metadata support)**, **Chroma (simple, developer-friendly)**.

What a vector database does

Scanning every vector for each query (“brute force”) is exact but doesn't scale. Vector DBs trade a sliver of accuracy for enormous speed.



Approximate search

ANN indexes like HNSW (graphs) or IVF (clustering) find neighbors in milliseconds across millions of vectors.



Metadata filtering

Combine vector similarity with structured filters — date, author, tenant, document type.



Scale & persistence

Sharding, replication and disk-backed storage so the index survives and grows.



Live updates

Insert, update and delete vectors without rebuilding the whole index.

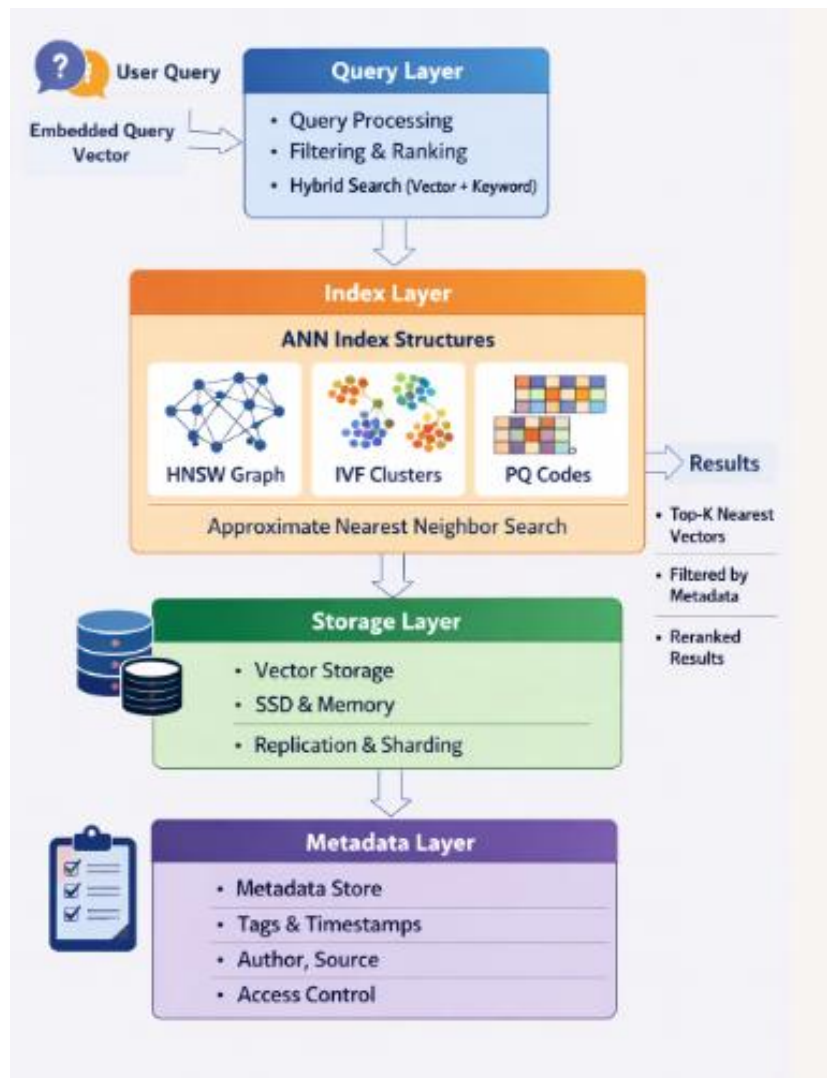
Why Were Vector Databases Created?

- Traditional databases work well for:
 - `SELECT * FROM employees
WHERE employee_id = 101;`
- or
 - `SELECT * FROM documents
WHERE title LIKE '%leave%'`
- These are:
 - Exact Match Search
Keyword Search

Why Were Vector Databases Created?

- But modern AI requires:
 - Semantic Search
 - Meaning-Based Search
- Example:
- Document:
 - Employees receive 20 annual leaves.
- User Query:
 - How many vacation days do employees get?
- No keyword match exists.
- Yet humans know:
 - vacation days $\approx$ annual leaves
- Vector databases solve this problem.

Vector Database Architecture



1. User Query Layer (Blue)

- Example query:
- How many annual leaves do employees get?
- The query is first converted into an embedding:
 - [0.12, -0.45, 0.78, ...]
- **Responsibilities**
 - **Query Processing**
 - Cleans and prepares query.
- Example:
 - How many annual leaves?
- becomes
 - annual leave policy



1. User Query Layer (Blue)

- **Filtering & Ranking**
- User wants:
 - Department = HR
 - Country = India
- The query layer applies filters before retrieval.



1. User Query Layer (Blue)

- **Hybrid Search**
- Combines:
 - Vector Search
 - +
Keyword Search
- Example:
- Query:
 - Azure Kubernetes Security
- Vector search finds:
 - AKS Security Guidelines
- Keyword search finds:
 - Azure Documentation
- Hybrid search merges both.

2. Index Layer (Search Engine)

- This is the most important layer.
- Without indexing:
 - 1 Million Documents
↓
Compare query with every vector
 - Very slow.
- Instead ANN indexes are used.

2. Index Layer (Search Engine)

- **HNSW Graph**
 - (Hierarchical Navigable Small World)
 - Most popular ANN algorithm.
- Think of:
 - Google Maps
- You don't check every road.
- You jump through major roads first.
- HNSW works similarly.
- **Advantages**
 - Very Fast
 - High Accuracy
 - Low Latency
- Used by:
 - Qdrant
 - Weaviate
 - OpenSearch Vector



2. Index Layer (Search Engine)

- **IVF Clusters**
- (Inverted File Index)
- Idea:
 - Group similar vectors
- Example:
 - HR Documents
 - Finance Documents
 - IT Documents
- Query:
 - Annual Leave
- Search only:
 - HR Cluster
- instead of all clusters.

2. Index Layer (Search Engine)

- **PQ Codes**
 - (Product Quantization)
- Problem:
 - 100 Million Vectors
 - Storage huge.
- Solution:
 - Compress vectors
- Example:
 - 1536 dimensions
 - ↓
 - Compressed representation
- Benefits:
 - Less RAM
 - Faster Search
- Trade-off:
 - Slightly lower accuracy

2. Index Layer (Search Engine)

- Approximate Nearest Neighbour Search (ANN)
- The Index Layer uses ANN.
- Goal:
 - Find Almost Best Match
 - Extremely Fast
- Instead of:
 - Perfect Match
 - Very Slow
- For enterprise systems:
 - 99% accuracy
 - 10ms latency
- is better than:
 - 100% accuracy
 - 5 seconds latency

3. Storage Layer (Physical Storage)

- Stores actual vectors.

- Example:

- Vector 1
- Vector 2
- Vector 3

...

-

- **Vector Storage**

- Stores:

- Embedding Arrays

- Example:

- [0.12, 0.45, ...]

3. Storage Layer (Physical Storage)

- **SSD & Memory**
- Frequently used vectors:
 - RAM
- Rarely used vectors:
 - SSD
- This improves performance.

3. Storage Layer (Physical Storage)

- **Replication**
 - Copies data.
- **Example:**
 - Node 1
 - Node 2
 - Node 3
- **If Node 1 fails:**
 - Node 2 continues serving

3. Storage Layer (Physical Storage)

- **Sharding**
 - Splits large data.
- Example:
 - 100 Million Vectors
 - Shard 1 → 25M
 - Shard 2 → 25M
 - Shard 3 → 25M
 - Shard 4 → 25M
- Enables horizontal scaling.

4. Metadata Layer

- Stores information about vectors.
- Not the vector itself.
- Example:
- ```
{
 "document": "Leave Policy",
 "department": "HR",
 "country": "India",
 "author": "HR Team",
 "year": "2026"
}
```



## 4. Metadata Layer

---

- **Metadata Store**
- Stores:
  - Document Name
  - Source
  - Department
  - Tags

## 4. Metadata Layer

---

- **Metadata Store**
- **Tags & Timestamps**
- **Example:**
  - Tag = Leave Policy
  - Created = 2026-01-01
  - Updated = 2026-05-01



## 4. Metadata Layer

---

- **Metadata Store**
- **Author & Source**
- Example:
  - Author = HR Team
  - Source = SharePoint

## 4. Metadata Layer

---

- **Access Control**
- Very important in enterprises.
- Example:
  - HR User
- Can access:
  - Leave Policies
- Cannot access:
  - Payroll Data
- Metadata filtering enforces security.

## 5. Results Layer

---

- After search:
  - Top-K Nearest Vectors

| Rank | Document          | Score |
|------|-------------------|-------|
| 1    | Leave Policy      | 0.95  |
| 2    | Employee Handbook | 0.89  |
| 3    | Travel Policy     | 0.32  |

## 5. Results Layer

---

- Then:
  - Top Documents
    - ↓
    - LLM
    - ↓
    - Final Answer
- Example:
- Question:

How many annual leaves do employees get?

Retrieved:  
Employees receive 20 annual leaves.

GPT Answer:  
Employees are entitled to 20 annual leave days annually.



## Interview Answer

---

- A vector database consists of four major layers: the Query Layer (processing and filtering), the Index Layer (ANN search using HNSW, IVF, PQ), the Storage Layer (vector persistence, replication, sharding), and the Metadata Layer (security, filtering, document attributes).
- Together they enable high-speed semantic search for RAG and AI applications.

# The vector store landscape



## The vector store landscape

| Option          | Shape               | Best when...                                                           |
|-----------------|---------------------|------------------------------------------------------------------------|
| <b>pgvector</b> | Postgres extension  | You already run Postgres and want vectors beside your relational data. |
| <b>Chroma</b>   | Embedded / local    | Prototyping, notebooks and small apps that need zero infra.            |
| <b>Qdrant</b>   | Open-source server  | Self-hosting with rich filtering and strong performance.               |
| <b>Weaviate</b> | Open-source + cloud | Hybrid search and a schema/module ecosystem out of the box.            |
| <b>Milvus</b>   | OSS, distributed    | Billions of vectors and heavy throughput at scale.                     |
| <b>Pinecone</b> | Fully managed SaaS  | You want serverless scale with no ops to run yourself.                 |

*Not exhaustive, and the space moves fast — match the tool to your scale, ops appetite and existing stack rather than chasing benchmarks.*

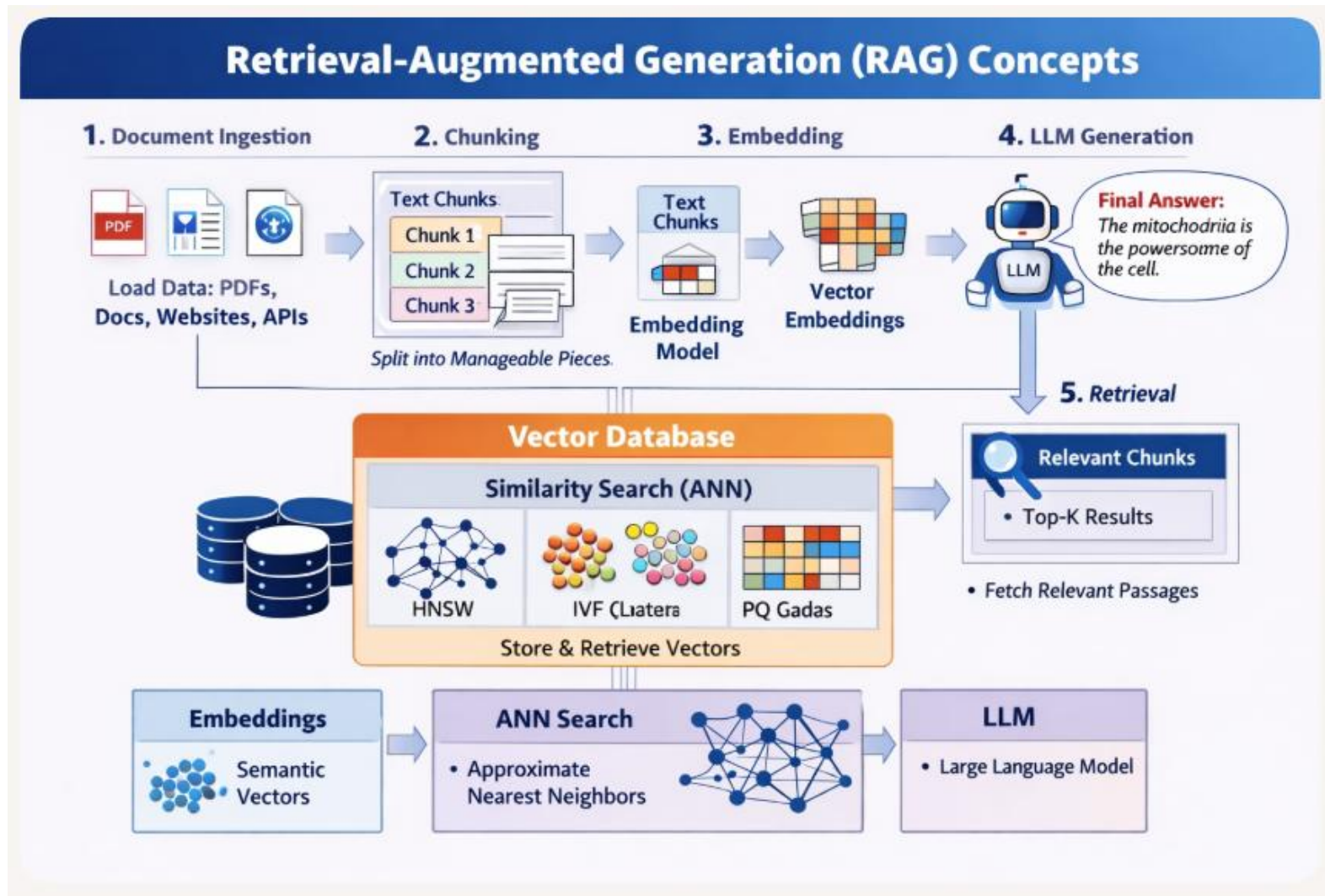


# What Problem Does RAG Solve?

---

- Suppose your company has:
  - 5000 PDFs
  - 10000 SOPs
  - HR Policies
  - Training Materials
  - Confluence Pages
- User asks:
  - How many annual leaves do employees get?
- An LLM alone doesn't know your company's policy.
- **RAG adds a retrieval layer before the LLM.**

# RAG Architecture





# RAG Architecture

---

- **Step 1: Document Ingestion**
- **Input Sources**
  - PDFs
  - Word Documents
  - Websites
  - APIs
  - Databases
  - Emails
- **Examples:**
  - HR Policy.pdf
  - Travel Policy.pdf
  - Security Standards.docx
- Documents are loaded into the RAG pipeline.

# RAG Architecture

---

- **Step 2: Chunking**
- **Why Chunking?**
  - A 100-page PDF cannot be sent entirely to an LLM.
- **Example:**
  - HR Policy.pdf
    - ↓
    - Chunk 1
    - Chunk 2
    - Chunk 3
    - Chunk 4
    - ...
- **Typical chunk size:**
  - 300–1000 words



# RAG Architecture

---

- Example:
- **Chunk 1**
  - Employees receive 20 annual leaves.
- **Chunk 2**
  - Employees receive medical insurance.
- **Chunk 3**
  - Travel reimbursement policy...

# RAG Architecture

---

- **Step 3: Embeddings**
  - Each chunk is converted into vectors.
- Example:
  - Employees receive 20 annual leaves.
- becomes:
  - [0.12, 0.45, -0.22, ...]
- using an embedding model.
- Popular models:
  - all-MiniLM-L6-v2 (384 dimensions)
  - BERT (768 dimensions)
  - OpenAI Embeddings (1536 dimensions)

# RAG Architecture

---

- **Step 4: Store in Vector Database**
  - The vectors are stored.

| Chunk               | Vector          |
|---------------------|-----------------|
| Annual Leave Policy | [0.12,0.45,...] |
| Medical Policy      | [0.77,0.23,...] |
| Travel Policy       | [0.55,0.81,...] |

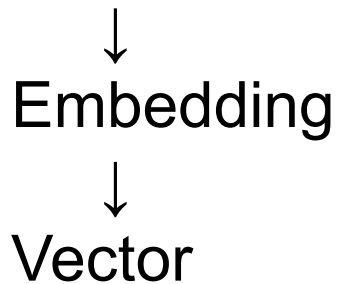
- **Stored in:**
  - FAISS
  - ChromaDB
  - Qdrant
  - Milvus

# RAG Architecture

---

- **Step 5: User Question**

- User asks:
- `How many annual leaves do employees get?
- This question is also converted into an embedding.
  - Question





# RAG Architecture

---

- **Step 6: Similarity Search (ANN)**
- The vector database performs:
- ANN
  - Approximate Nearest Neighbor Search
- instead of scanning every document.
- The diagram shows 3 ANN techniques:
- **HNSW**
  - Graph-based Search
- Fastest and most popular.



# RAG Architecture

---

- **IVF**
  - Cluster-based Search
  - Searches only relevant clusters.
- **PQ**
  - Compressed Search
  - Used for massive datasets.

# RAG Architecture

---

- **Step 7: Retrieval**
- Vector database returns:
  - Top-K Relevant Chunks

| Rank | Chunk                              |
|------|------------------------------------|
| 1    | Employees receive 20 annual leaves |
| 2    | Leave encashment policy            |
| 3    | Employee handbook                  |



# RAG Architecture

---

- **Step 8: LLM Generation**

- Retrieved chunks are sent to GPT/Claude.

- Prompt becomes:

- Context:

- Employees receive 20 annual leaves.

- Question:

- How many annual leaves do employees get?

- LLM generates:

- Employees are entitled to 20 annual leave days annually.

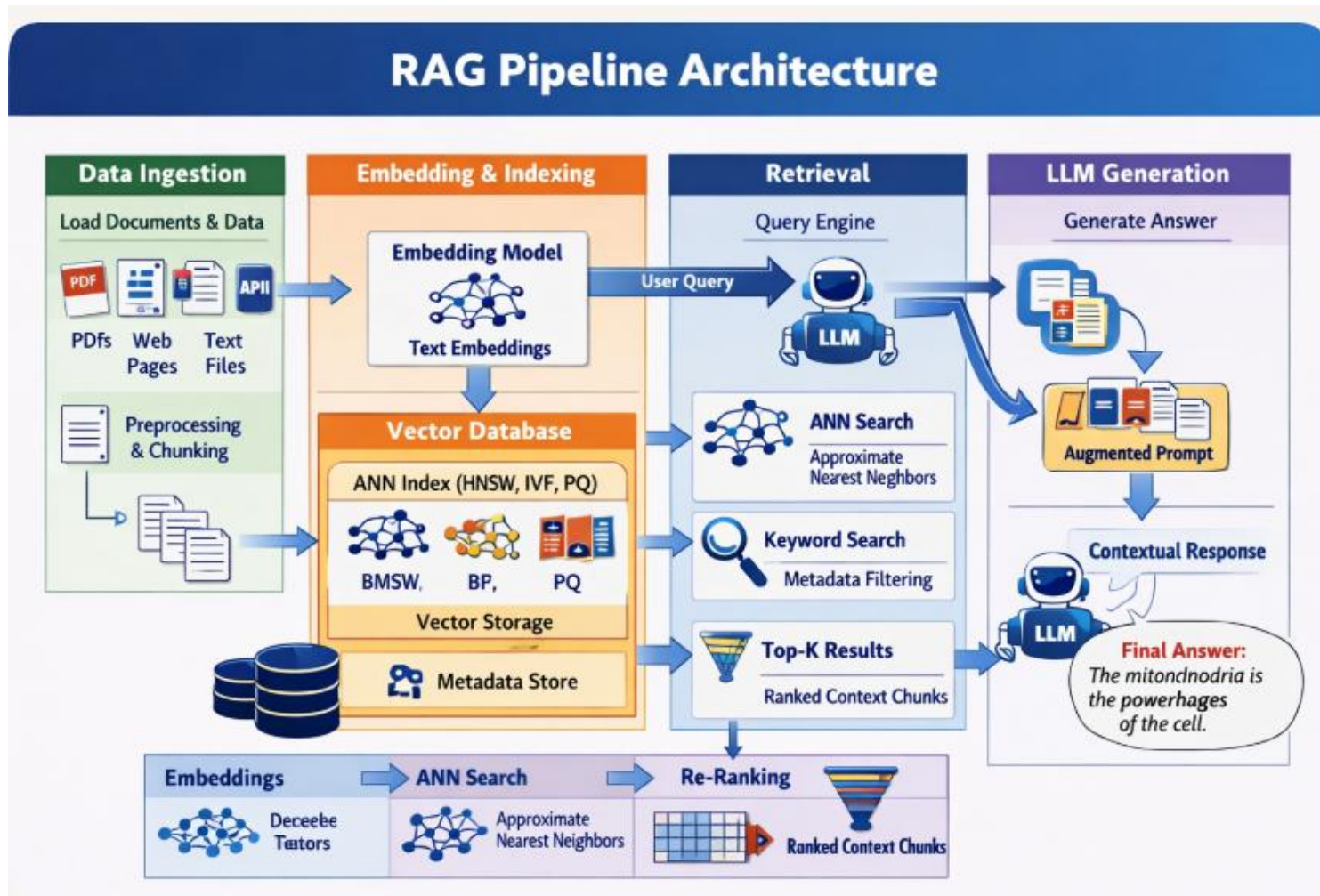


## Interview Answer

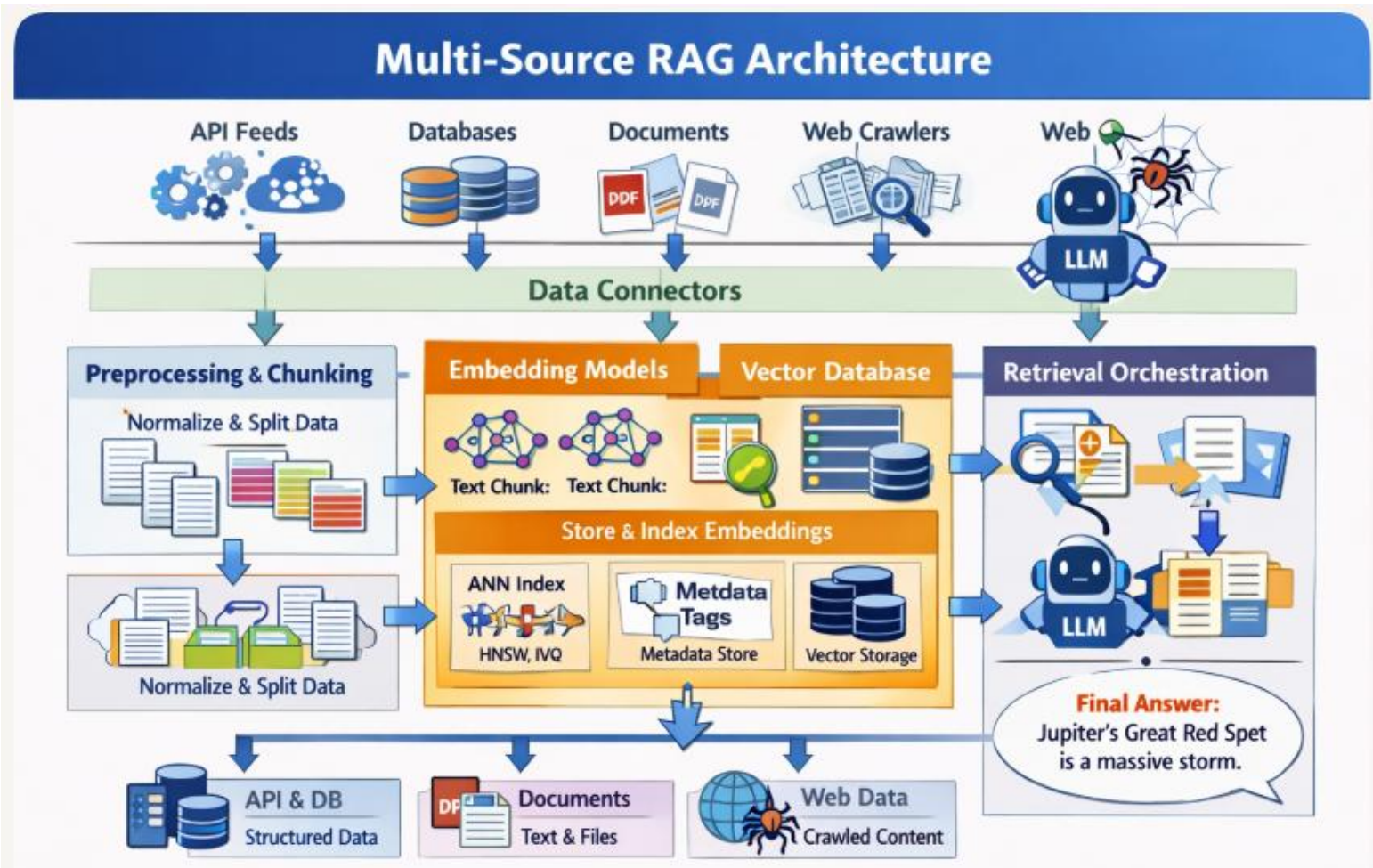
---

- **Explain the RAG architecture.**
- RAG (Retrieval-Augmented Generation) combines document retrieval with LLM generation.
- Documents are ingested, chunked, converted into embeddings, and stored in a vector database.
- When a user asks a question, the query is embedded and used to perform similarity search.
- The most relevant chunks are retrieved and provided to the LLM as context, enabling accurate and up-to-date answers while reducing hallucinations.

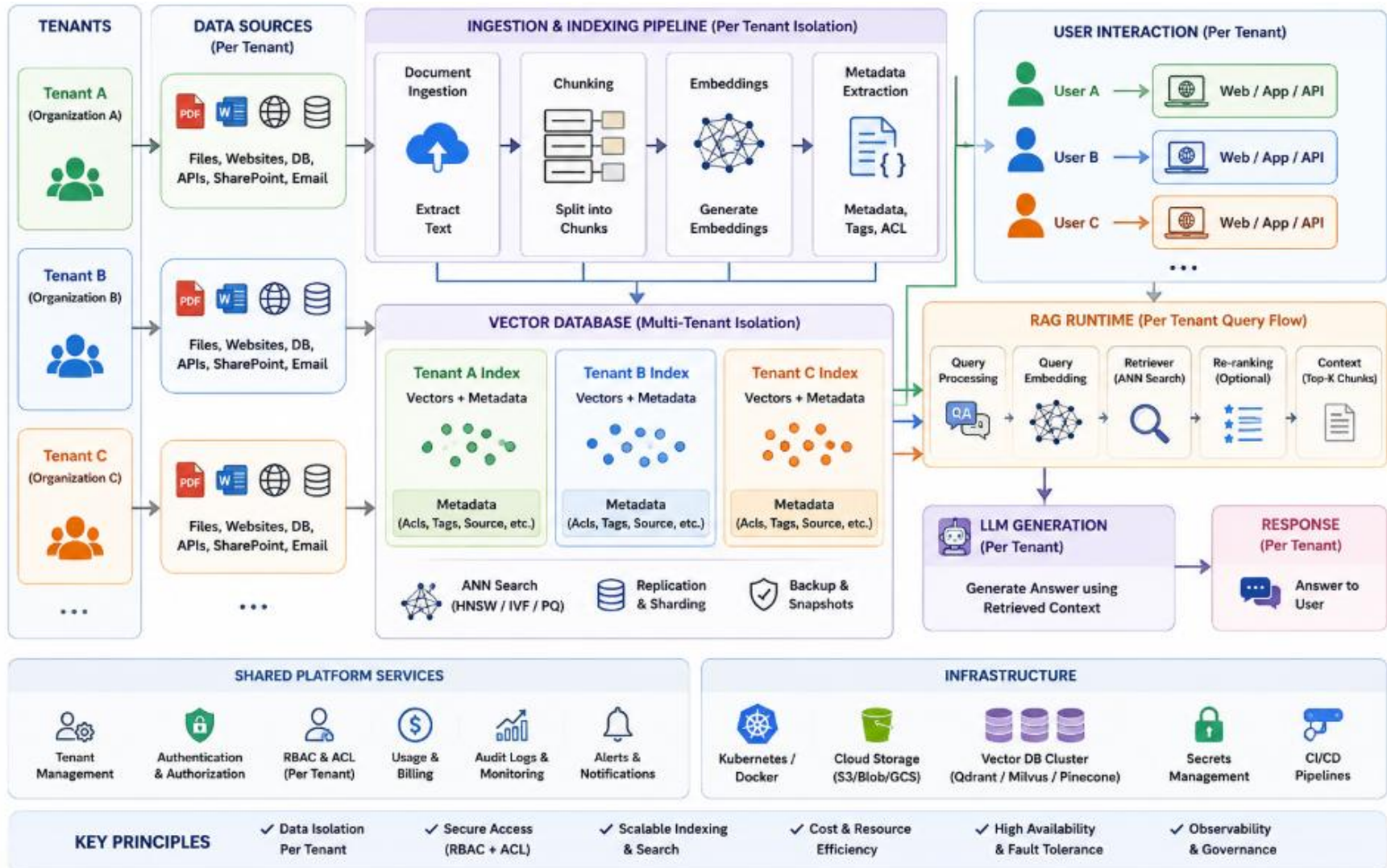
# RAG pipeline architecture



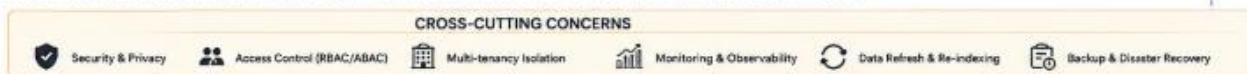
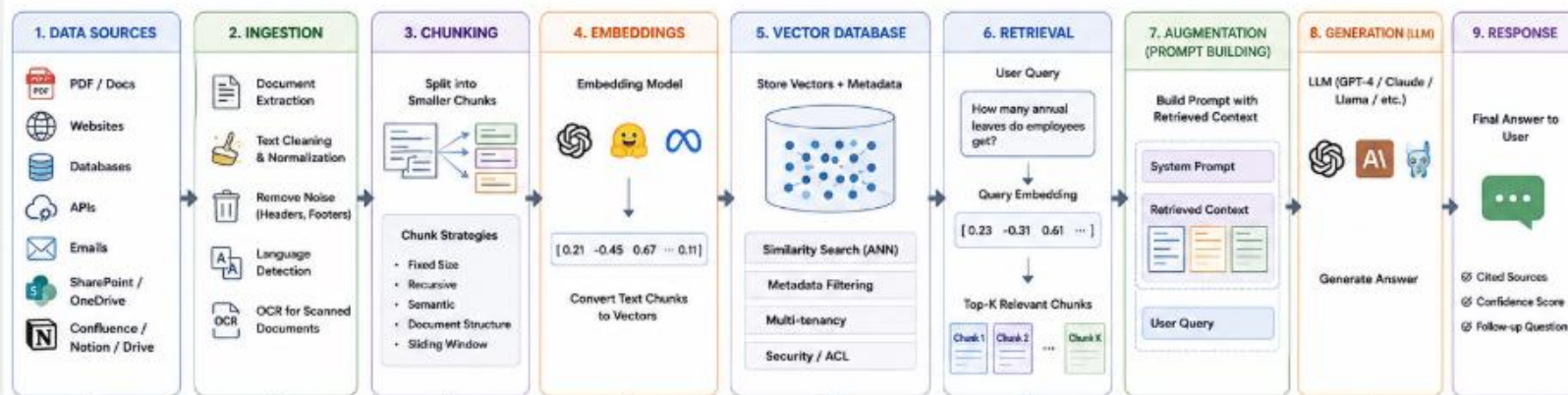
# Multi Source RAG



# Multi-tenant RAG Architecture



# Real time Use Cases



## REAL TIME RAG USE CASES



### KEY BENEFITS OF RAG



# SageMaker



- Amazon SageMaker is AWS's managed platform for building, training, tuning, deploying, and monitoring machine-learning and generative-AI solutions.





- **SageMaker Studio** – development environment for notebooks and ML workflows.
- **Data processing** – clean and transform datasets using managed processing jobs.
- **Training jobs** – train built-in algorithms or custom Scikit-learn, TensorFlow and PyTorch models.
- **Automatic Model Tuning** – identifies suitable hyperparameters.
- **Model Registry** – versions and approves models before deployment.

- **Endpoints** – provides real-time prediction APIs.
- **Batch Transform** – generates offline predictions for large datasets.
- **Pipelines** – automates data preparation, training, evaluation and deployment.
- **Model Monitor** – detects data quality problems and model drift.
- **Clarify** – examines bias and explains predictions.
- **Canvas** – supports visual, low-code model development.



# Why SageMaker

---

- **Less infrastructure management**  
AWS provisions training servers, storage integration and model endpoints.
- **End-to-end ML workflow**  
Prepare data → train → evaluate → deploy → monitor.
- **Easy AWS integration**  
Works directly with S3, Glue, Athena, Redshift, Lambda, API Gateway and CloudWatch.
- **Scalable training**  
Start with a small instance and scale to distributed CPU or GPU training.
- **Simple model deployment**  
Deploy a trained model as a secure HTTPS API endpoint.
- **Multiple development options**  
Supports notebooks, Python SDK, built-in algorithms, AutoML and no-code SageMaker Canvas.



## Why SageMaker

---

- **MLOps support**

SageMaker Pipelines and Model Registry provide automated workflows, model versioning and approval.

- **Model monitoring**

Detects data drift, prediction-quality degradation and bias after deployment.

- **Pay-as-you-use**

Training resources can stop when the job completes. However, real-time endpoints continue generating charges until deleted.

## With or Without SageMaker

---

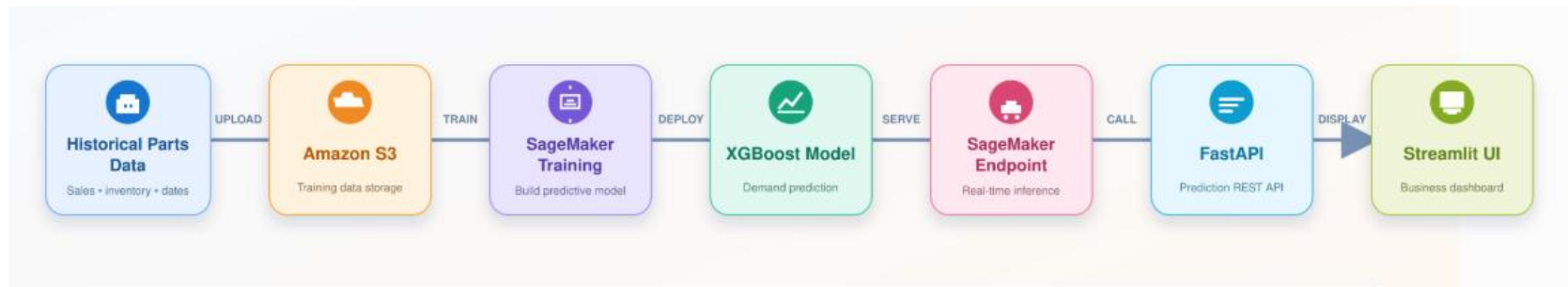
| Activity                       | Without SageMaker               | With SageMaker                |
|--------------------------------|---------------------------------|-------------------------------|
| Provision servers              | Manual                          | Managed                       |
| Configure training environment | Manual                          | Prebuilt or custom containers |
| Hyperparameter tuning          | Custom implementation           | Managed tuning jobs           |
| Deploy prediction API          | Build and manage infrastructure | Managed endpoint              |
| Scale predictions              | Configure manually              | Managed auto scaling          |
| Track model versions           | Build your own system           | Model Registry                |
| Monitor drift                  | Custom solution                 | Model Monitor                 |

## Project objective

---

- Given these inputs:
  - Part category
  - Current stock
  - Previous-month demand
  - Average monthly demand
  - Vehicle sales
  - Seasonal index
  - Supplier lead time
- Predict:
  - Next-month spare-part demand

# Project objective





# AWS services and software

---

| Component          | Purpose                                 |
|--------------------|-----------------------------------------|
| Amazon S3          | Store training data and model artifacts |
| SageMaker Studio   | Development environment                 |
| SageMaker Training | Train the XGBoost model                 |
| SageMaker Endpoint | Real-time predictions                   |
| CloudWatch         | Training and endpoint logs              |
| Python             | Data preparation and application        |
| FastAPI            | Prediction REST API                     |
| Streamlit          | User interface                          |



## Terraform Overview

---



- Terraform is an **Infrastructure as Code (IaC)** tool that lets you define, provision, and manage infrastructure across multiple cloud providers (Azure, AWS, GCP, etc.) using a declarative language.
- It ensures **consistency, repeatability, and automation** in deployments.

# What is Infrastructure as Code?

Instead of clicking buttons in a cloud portal, you describe the servers, storage and networks you want in a text file. A tool then builds them for you — the same way, every time.



## Manual (Click-Ops)

- Click through portal screens
- Hard to repeat exactly
- No record of what changed
- Slow & error-prone at scale



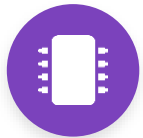
## Infrastructure as Code

- ✓ Write it once in a file
- ✓ Run it any number of times
- ✓ Tracked in version control (Git)
- ✓ Fast, consistent, reviewable

# Why ML Teams Need IaC



*Machine learning needs powerful, expensive, fast-changing infrastructure. IaC keeps it under control.*



## Expensive GPUs

Spin up GPU clusters only for training, then tear them down to save money.



## Reproducibility

The exact same environment every run — no "works on my machine".



## Scale on demand

Go from 1 to 100 machines for a big job with one config change.



## Cost control

See every resource in code; avoid forgotten servers burning cash.

# Meet the Two Tools



## THE TOOLKIT



### Terraform

#### MAKER

HashiCorp (open source)

#### WORKS ON

AWS, Azure, GCP & 100s more

#### LANGUAGE

HCL — HashiCorp Config Language

#### BEST FOR

Multi-cloud, large teams



### Bicep

#### MAKER

Microsoft

#### WORKS ON

Azure only

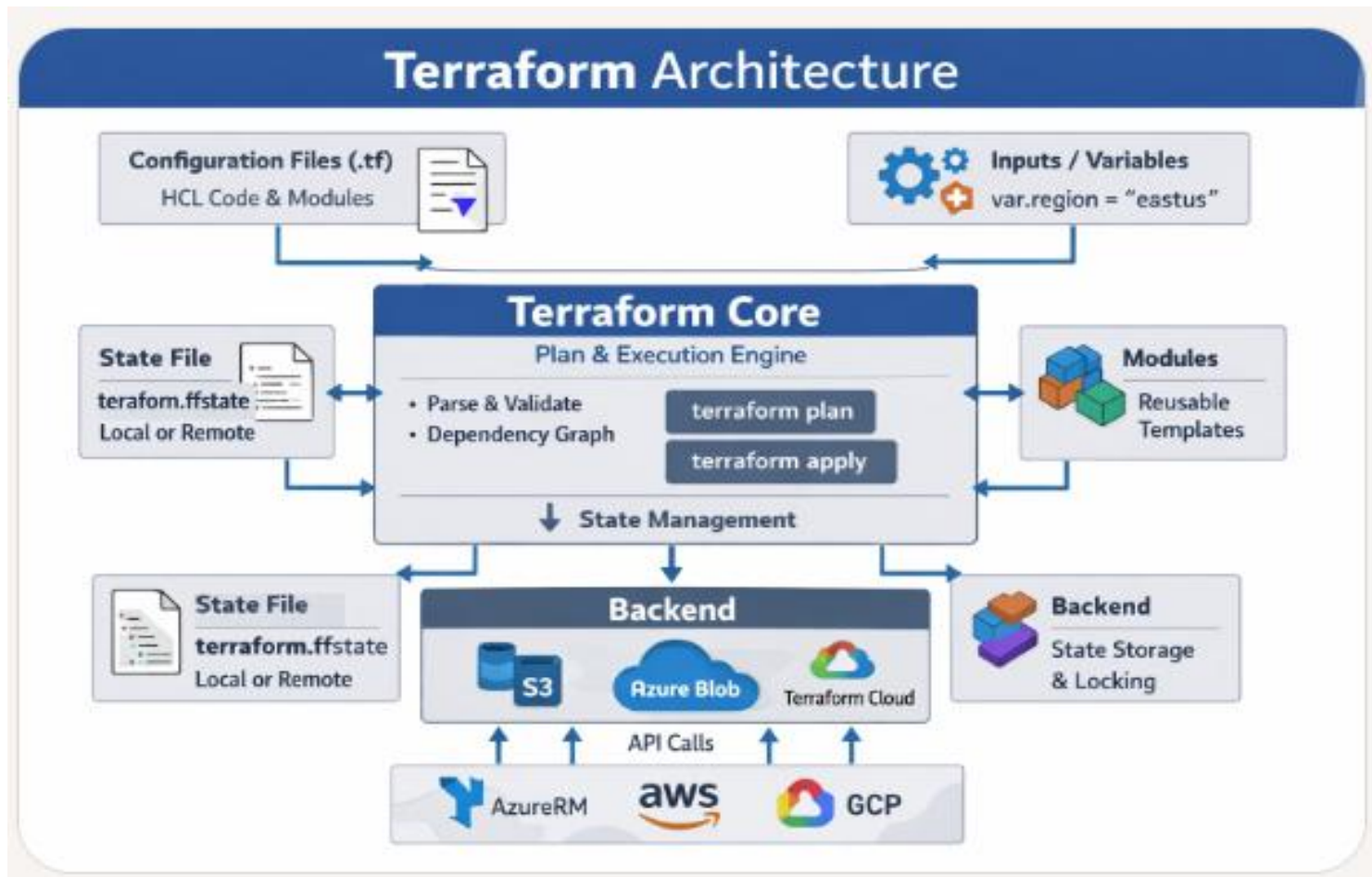
#### LANGUAGE

Bicep — a clean Azure DSL

#### BEST FOR

Azure-focused teams

# Terraform Architecture



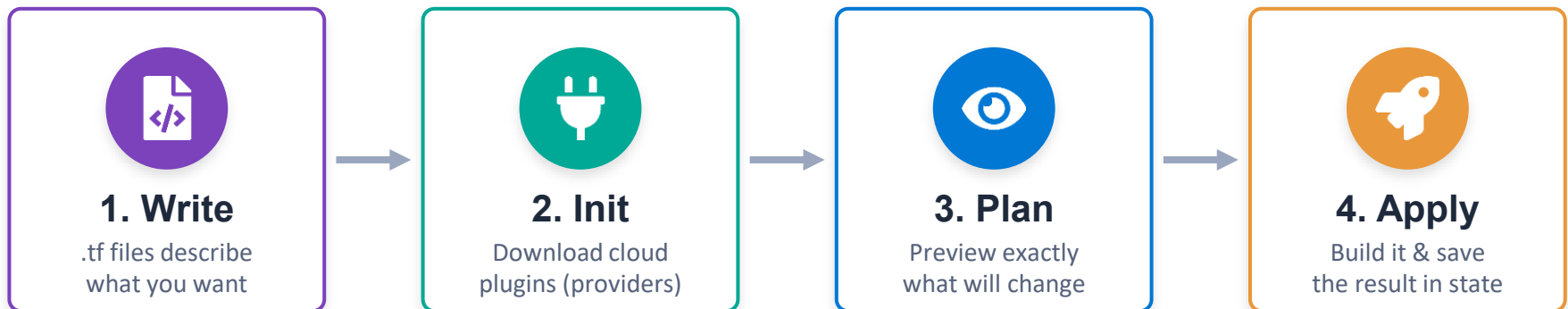
-  **Core Layers**
- **Configuration Files**
  - Written in HCL (.tf files).
  - Define providers, resources, variables, outputs, and modules.
- **Terraform Core**
  - The engine that parses configs, builds dependency graphs, and executes plans.
  - Handles state management and communicates with providers.

-  **Core Layers**
- **Backend**
  - Defines where state is stored and how operations are executed.
  - Examples: local, remote, or cloud backends with locking.
- **Modules**
  - Reusable building blocks grouping multiple resources.
  - Promote consistency across environments (Dev, Test, Prod).

# How Terraform Works

## TERRAFORM WORKFLOW

*Four commands take you from a text file to live cloud infrastructure.*



**The State File:** Terraform remembers what it built in a **state file**. On the next run it compares your code to reality and changes only the difference.

# Terraform Core Concepts



## BUILDING BLOCKS



### Provider

A plugin for a cloud (AWS, Azure, GCP). It knows how to create that cloud's resources.



### Resource

One thing you want to exist — a VM, a bucket, a database, a network.



### Variable

An input you can change without editing code — region, machine size, count.

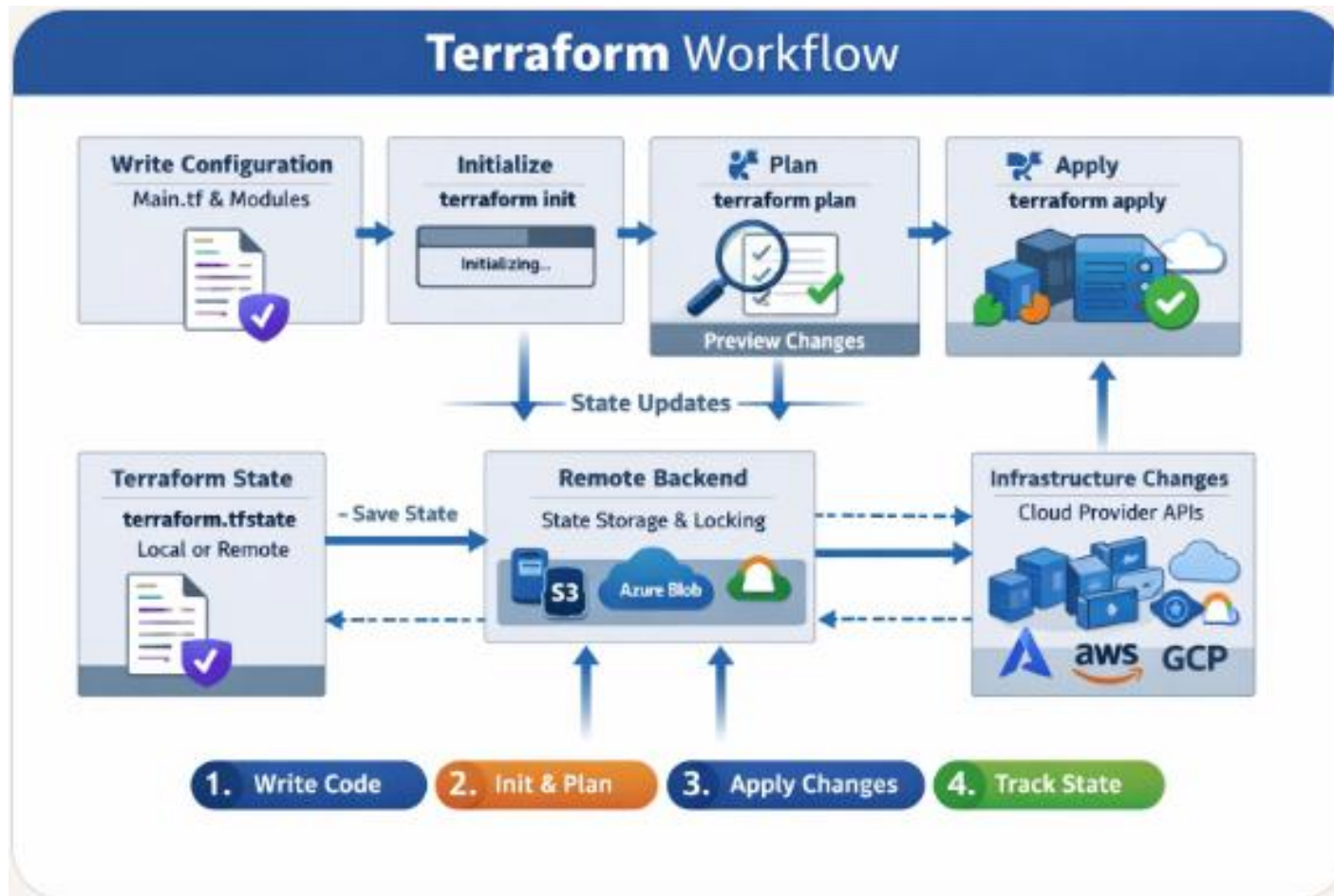


### Module

A reusable bundle of resources you can call again and again, like a function.



# Workflow in Architecture



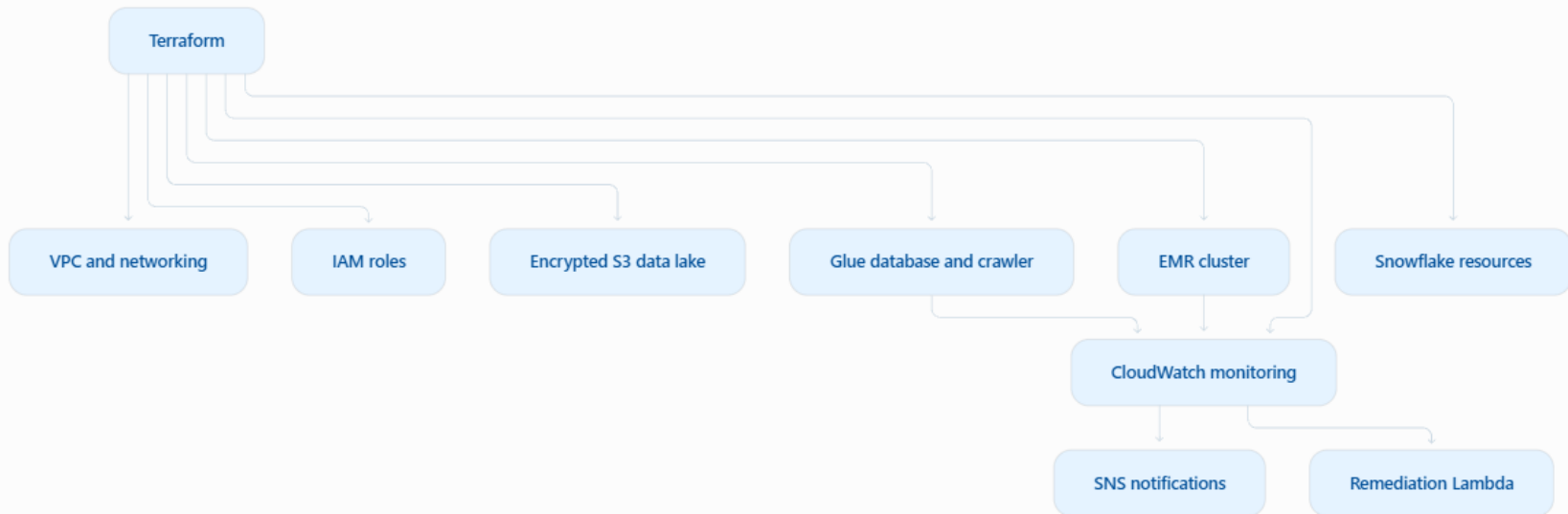


## Workflow in Architecture



- **Write Configuration** → Developers define resources in .tf files using HCL.
- **Initialize** → terraform init downloads provider plugins and sets up the backend.
- **Plan** → terraform plan previews what changes will occur before applying.
- **Apply** → terraform apply provisions resources via cloud APIs.
- **State Updates** → Terraform records the new infrastructure in the state file.
- **Backend** → Stores state remotely (Azure Blob, S3, Terraform Cloud) for collaboration and locking.
- **Infrastructure Changes** → Cloud providers (Azure, AWS, GCP) execute API calls to create or modify resources.

# Terraform Use Cases



# Terraform

---

- **1. Prerequisites**
- Install:
  - Terraform CLI
  - AWS CLI
  - An AWS account with suitable permissions
  - Azure CLI for the Azure backend lab
  - Snowflake account for the Snowflake lab
  - Graphviz for resource graph visualization
- Verify:
  - terraform version
  - aws --version
  - aws sts get-caller-identity
- Configure AWS:
  - aws configure



- **Run Terraform**
  - terraform fmt -recursive
  - terraform init
  - terraform validate
  - terraform plan
  - terraform apply

# Terraform vs Bicep



|                       | Terraform                      | Bicep                   |
|-----------------------|--------------------------------|-------------------------|
| <b>Cloud support</b>  | Any cloud (AWS, Azure, GCP...) | Azure only              |
| <b>Made by</b>        | HashiCorp                      | Microsoft               |
| <b>Language</b>       | HCL                            | Bicep DSL               |
| <b>State file</b>     | Yes — you manage it            | No — Azure tracks it    |
| <b>Learning curve</b> | Gentle, lots of examples       | Very easy on Azure      |
| <b>Pick it when</b>   | You use many clouds            | You are all-in on Azure |

# Best Practices for Beginners

## HABITS THAT KEEP YOU SAFE



### Never hardcode secrets

Keep keys & passwords out of code; use a vault or variables.



### Commit to Git

Version every change so you can review and roll back.



### Always read the plan

Preview changes before applying — avoid surprise deletions.



### Reuse with modules

Write once, reuse everywhere; keep configs small and clean.



### Start small

Build one resource, confirm it works, then grow step by step.



### Tear down unused infra

Destroy idle GPU clusters so the bill stays low.